



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS
COMPUTACIONALES
DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE
SISTEMAS

Desarrollo de Software IX

Diseño: Historias de Usuario MVP — Production-Ready y
MCP

Profesor: Erick Agrazal

Integrantes: Ivan Wu 8-960-1362
Adrian Wong 8-958-1862
Gabriel Cubilla4-777-2220

Fecha: 6/23/26

Índice - Historias de Usuario Dolce Atelier.....	5
Sección 1: Historias Implementadas (MVP Actual).....	5
Sección 2: Historias Production-Ready (Nuevas).....	7
Resumen Global.....	9
Herramientas MCP (Detalle).....	10
Historias Implementadas.....	10
HU-001 [MCP].....	10
HU-002 [MCP].....	10
HU-003.....	11
HU-004.....	11
HU-005.....	11
HU-006.....	12
HU-007.....	12
HU-008 [MCP].....	12
HU-009.....	13
HU-010.....	13
HU-011.....	13
HU-012.....	14
HU-013.....	14
HU-014.....	14
HU-015.....	15
HU-016 [MCP].....	15
HU-017 [MCP].....	15
HU-018.....	16
HU-019 [MCP].....	16
HU-020 [MCP].....	17
HU-021 [MCP].....	17
HU-022 [MCP].....	17
HU-023.....	18
HU-024 [MCP].....	18
HU-025 [MCP].....	18
HU-026 [MCP].....	19
HU-027.....	19
HU-028 [MCP].....	19
HU-029.....	20
HU-030 [MCP].....	20
HU-031.....	20
HU-032.....	21

HU-033.....	21
HU-034.....	21
HU-035.....	22
HU-036 [MCP].....	22
HU-037 [MCP].....	22
HU-038.....	23
HU-039.....	23
HU-040.....	23
Historias Production Ready.....	24
HU-041.....	24
HU-042.....	24
HU-043.....	25
HU-044.....	25
HU-045.....	25
HU-046.....	26
HU-047.....	26
HU-048.....	26
HU-049.....	27
HU-050.....	27
HU-051.....	27
HU-052.....	28
HU-053.....	28
HU-054.....	29
HU-055.....	29
HU-056.....	29
HU-057.....	30
HU-058.....	30
HU-059.....	30
HU-060.....	31
HU-061.....	31
HU-062.....	31
HU-063.....	32
HU-064.....	32
HU-065.....	32
HU-066.....	33
HU-067.....	33
HU-068.....	34
HU-069.....	34

HU-070.....	34
HU-071.....	35
HU-072.....	35
HU-073.....	35
HU-074.....	36
HU-075.....	36
HU-076.....	36
HU-077.....	37
HU-078.....	37
HU-079.....	37
HU-080.....	38
Catálogo de Herramientas MCP - Dolce Atelier.....	38
Definición MCP.....	38
Autenticación MCP.....	39
Tool: `search_cakes`.....	39
Tool: `get_cake_detail`.....	41
Tool: `add_to_cart`.....	42
Tool: `get_cart`.....	43
Tool: `create_order`.....	45
Tool: `list_orders`.....	47
Tool: `get_order`.....	48
Tool: `submit_recipe`.....	49
Tool: `list_my_recipes`.....	51
Tool: `accept_quote`.....	52
Tool: `admin_list_cakes`.....	53
Tool: `admin_update_order_status`.....	55
Tool: `admin_quote_recipe`.....	56
Tool: `admin_get_dashboard_stats`.....	57
Tool: `handle_stripe_webhook`.....	59
Tool: `admin_search_users`.....	60
Tool: `export_orders_csv`.....	61
Tool: `health_check`.....	63
Tool: `admin_export_pasteles`.....	64
Mapeo: Historias ↔ Tool MCP ↔ Auth ↔ Capability.....	65
Tabla de Mapeo MCP.....	65
Resumen por Nivel de Auth.....	66
Ejemplo de Uso en Claude Code.....	67
Ejemplo de Uso en Codex.....	68

Flujo de Autenticación MCP.....	69
Resumen Ejecutivo - Historias de Usuario Dolce Atelier.....	69
Contexto.....	69
Estado Actual del MVP.....	70
 Implementado (40 historias).....	70
 No implementado (excluido del MVP).....	71
Production-Ready: Lo que Falta (40 historias nuevas).....	71
MCP: Herramientas para Agentes IA (19 tools).....	72
Concepto.....	72
Herramientas Definidas.....	72
Autenticación MCP.....	73
Ejemplo en Claude Code.....	73
Criterios de Validación.....	73
Próximos Pasos.....	74

Índice - Historias de Usuario Dolce Atelier

Sección 1: Historias Implementadas (MVP Actual)

#	Título	MCP	Archivo
HU-001	Buscar y listar pasteles del catálogo	[MCP]	01-implementadas/HU-001-buscar-listar-pasteles.md
HU-002	Ver detalle completo de un pastel	[MCP]	01-implementadas/HU-002-ver-detalle-pastel.md
HU-003	Ver categorías en página de inicio		01-implementadas/HU-003-ver-categorias-inicio.md
HU-004	Agregar pasteles al carrito	[MCP]	01-implementadas/HU-004-agregar-al-carrito.md
HU-005	Modificar cantidades en carrito		01-implementadas/HU-005-modificar-cantidades-carrito.md
HU-006	Eliminar items del carrito		01-implementadas/HU-006-eliminar-items-carrito.md
HU-007	Persistencia y fusión del carrito		01-implementadas/HU-007-persistencia-fusion-carrito.md
HU-008	Ver carrito con totales	[MCP]	01-implementadas/HU-008-ver-carrito-totales.md
HU-009	Seleccionar método de entrega		01-implementadas/HU-009-seleccionar-metodo-entrega.md
HU-010	Ingresar datos de contacto		01-implementadas/HU-010-ingresar-datos-contacto.md
HU-011	Enviar pedido al backend (Stripe)		01-implementadas/HU-011-crear-pedido-stripe.md
HU-012	Redirigir a Stripe Checkout		01-implementadas/HU-012-redirigir-stripe-checkout.md
HU-013	Página de éxito post-pago		01-implementadas/HU-013-pagina-exito-pago.md

HU-014	Página de error post-pago		01-implementadas/HU-014-pagina-error-pago.md
HU-015	Webhook Stripe confirmar pago	[MCP]	01-implementadas/HU-015-webhook-stripe-confirmar-pago.md
HU-016	Ver historial de pedidos	[MCP]	01-implementadas/HU-016-ver-historial-pedidos.md
HU-017	Ver detalle de un pedido	[MCP]	01-implementadas/HU-017-ver-detalle-pedido.md
HU-018	Página de recetas (placeholder)		01-implementadas/HU-018-pagina-recetas-placeholder.md
HU-019	Enviar receta personalizada	[MCP]	01-implementadas/HU-019-enviar-receta-personalizada.md
HU-020	Ver mis recetas con estados	[MCP]	01-implementadas/HU-020-ver-mis-recetas.md
HU-021	Aceptar cotización y pagar	[MCP]	01-implementadas/HU-021-aceptar-cotizacion-receta.md
HU-022	Admin dashboard con KPIs	[MCP]	01-implementadas/HU-022-admin-dashboard-kpis.md
HU-023	Admin listar pasteles	[MCP]	01-implementadas/HU-023-admin-listar-pasteles.md
HU-024	Admin crear pastel	[MCP]	01-implementadas/HU-024-admin-crear-pastel.md
HU-025	Admin editar pastel	[MCP]	01-implementadas/HU-025-admin-editar-pastel.md
HU-026	Admin eliminar pastel	[MCP]	01-implementadas/HU-026-admin-eliminar-pastel.md
HU-027	Admin listar pedidos	[MCP]	01-implementadas/HU-027-admin-listar-pedidos.md
HU-028	Admin actualizar estado pedido	[MCP]	01-implementadas/HU-028-admin-actualizar-estado-pedido.md
HU-029	Admin listar recetas	[MCP]	01-implementadas/HU-029-admin-listar-recetas.md
HU-030	Admin cotizar receta	[MCP]	01-implementadas/HU-030-admin-cotizar-receta.md

HU-031	Auth Clerk (SSO, Magic Link, OTP)		01-implementadas/HU-031-auth-clerk-sso-magiclink-otp.md
HU-032	Protección de rutas (middleware)		01-implementadas/HU-032-proteccion-rutas-middleware.md
HU-033	Backend verificar token Clerk		01-implementadas/HU-033-backend-verificar-token-clerk.md
HU-034	Backend verificar rol admin		01-implementadas/HU-034-backend-verificar-rol-admin.md
HU-035	Seed datos de prueba		01-implementadas/HU-035-seed-datos-prueba.md
HU-036	Health check endpoint	[MCP]	01-implementadas/HU-036-health-check-endpoint.md
HU-037	Upload imágenes a Cloudinary	[MCP]	01-implementadas/HU-037-upload-imagenes-cloudinary.md
HU-038	Upload archivo receta		01-implementadas/HU-038-upload-archivo-receta.md
HU-039	Email factura post-pago		01-implementadas/HU-039-email-factura-post-pago.md
HU-040	Script dev orquestador		01-implementadas/HU-040-script-dev-orquestador.md

Total Implementadas: 40 historias (21 con tag [MCP])

Sección 2: Historias Production-Ready (Nuevas)

#	Título	MCP	Categoría
HU-041	Rate limiting API	[MCP]	Seguridad
HU-042	JWT refresh rotation		Seguridad
HU-043	CSP headers		Seguridad
HU-044	Auditoría accesos	[MCP]	Seguridad
HU-045	Secret scanning CI/CD		Seguridad
HU-046	Idempotencia pedidos/webhooks	[MCP]	Pagos
HU-047	Reintentos webhook Stripe	[MCP]	Pagos

HU-048	Reembolsos Stripe	[MCP]	Pagos
HU-049	PCI DSS SAQ-A		Pagos
HU-050	Logging estructurado	[MCP]	Observabilidad
HU-051	Métricas Prometheus	[MCP]	Observabilidad
HU-052	Distributed tracing OTEL	[MCP]	Observabilidad
HU-053	Alertas PagerDuty		Observabilidad
HU-054	Health checks profundos	[MCP]	Observabilidad
HU-055	Error Boundaries React		Errores
HU-056	Manejo errores global	[MCP]	Errores
HU-057	Dead Letter Queue	[MCP]	Errores
HU-058	Tests unitarios Vitest		Testing
HU-059	Tests integración Testcontainers		Testing
HU-060	Tests E2E Playwright		Testing
HU-061	Contract testing Pact	[MCP]	Testing
HU-062	Mutation testing Stryker		Testing
HU-063	CI/CD GitHub Actions		CI/CD
HU-064	Preview deployments		CI/CD
HU-065	Blue-green deployment		CI/CD
HU-066	Rollback automatizado		CI/CD
HU-067	RBAC granular	[MCP]	RBAC
HU-068	Admin UI roles	[MCP]	RBAC
HU-069	Impersonación admin	[MCP]	RBAC
HU-070	ABAC recetas	[MCP]	RBAC
HU-071	Anti-brute force		Rate Limiting
HU-072	MCP rate limiting API keys	[MCP]	Rate Limiting
HU-073	MongoDB PITR		Backups
HU-074	Seed recovery script		Backups

HU-075	RTO/RPO runbooks	Backups
HU-076	WCAG 2.1 AA	Accesibilidad
HU-077	Navegación teclado	Accesibilidad
HU-078	ARIA live regions	Accesibilidad
HU-079	i18n next-intl	i18n
HU-080	CSS RTL-ready	i18n

Total Production-Ready: 40 historias nuevas (15 con tag [MCP])

Resumen Global

Métrica	Valor
Total historias	80
Implementadas	40
Production-Ready (nuevas)	40
Con tag [MCP]	36 (21 implementadas + 15 production-ready)
Herramientas MCP definidas	19
Requisito mínimo 30 nuevas	✓ 40
Requisito mínimo 15 MCP	✓ 36

Herramientas MCP (Detalle)

Ver `03-mcp-catalogo/TOOLS.md` para schema JSON completo de cada tool.

Ver `04-mapeo-mcp/MAPEO.md` para tabla de mapeo Historia ↔ Tool ↔ Auth ↔ Capability.

Historias Implementadas

HU-001 [MCP]

Como cliente visitante, **quiero** buscar y listar pasteles del catálogo con filtros, **para** encontrar el pastel que deseo comprar.

Criterios de Aceptación:

1. **Dado** que estoy en la página de catálogo **cuando** cargo la página **entonces** veo una lista paginada de pasteles (12 por página) renderizada en SSR
2. **Dado** que estoy en el catálogo **cuando** selecciono una categoría (chocolate, vainilla, frutas, especial) **entonces** la lista se filtra solo a pasteles de esa categoría
3. **Dado** que hay más de 12 pasteles **cuando** navego a la página 2 **entonces** veo los siguientes pasteles con paginación funcional
4. **Dado** que busco pasteles **cuando** la respuesta del backend incluye `total`, `page`, `totalPages` **entonces** la paginación se calcula correctamente

Etiquetas: [MCP] | Tool: `search\_cakes` | Auth: `API Key + User JWT`

HU-002 [MCP]

Como cliente visitante, **quiero** ver el detalle completo de un pastel, **para** evaluar su descripción, precio e imagen antes de agregarlo al carrito.

Criterios de Aceptación:

1. **Dado** que estoy en el catálogo **cuando** hago clic en un pastel **entonces** navego a `/catalogo/[id]` con SSR del detalle
2. **Dado** que estoy en la página de detalle **cuando** cargo la página **entonces** veo nombre, descripción, precio, imagen (Cloudinary), categoría y disponibilidad
3. **Dado** que el pastel no está disponible **cuando** veo el detalle **entonces** no puedo agregarlo al carrito (botón deshabilitado)
4. **Dado** que el pastel no existe **cuando** accedo a un ID inválido **entonces** veo un error 404 manejado

Etiquetas: [MCP] | Tool: `get\_cake\_detail` | Auth: `API Key + User JWT`

HU-003

Como cliente visitante, **quiero** ver las categorías de pasteles en la página de inicio, **para** navegar rápidamente al catálogo filtrado por mi preferencia.

Criterios de Aceptación:

1. **Dado** que estoy en la página de inicio **cuando** cargo la página **entonces** veo 4 tarjetas de categorías (Chocolate, Vainilla, Frutas, Personalizado) con imágenes
2. **Dado** que hago clic en una categoría **cuando** navego **entonces** voy a `/catalogo?categoria=<slug>` con el filtro aplicado
3. **Dado** que la página carga **cuando** inspecciono el HTML **entonces** el contenido de categorías está renderizado en SSR (SEO)
4. **Dado** que estoy en móvil **cuando** veo las categorías **entonces** se muestran en grid 2x2 responsive

Etiquetas:

HU-004

Como cliente, **quiero** agregar pasteles a mi carrito de compras, **para** acumular mi pedido antes de proceder al checkout.

Criterios de Aceptación:

1. **Dado** que estoy en el catálogo o detalle **cuando** hago clic en "Agregar" **entonces** el pastel se añade al estado global (Zustand) con cantidad 1
2. **Dado** que agrego el mismo pastel múltiples veces **cuando** reviso el carrito **entonces** la cantidad se incrementa (no se duplica la línea)
3. **Dado** que agrego items **cuando** navego a `/carrito` **entonces** veo todos los items con imagen, nombre, precio unitario, cantidad y subtotal
4. **Dado** que cierro el navegador **cuando** vuelvo a abrir **entonces** el carrito persiste en localStorage

Etiquetas: [MCP] | Tool: `add\_to\_cart` | Auth: `User JWT`

HU-005

Como cliente, **quiero** modificar las cantidades de items en mi carrito, **para** ajustar mi pedido antes de pagar.

Criterios de Aceptación:

1. **Dado** que estoy en `/carrito` **cuando** uso los botones +/- **entonces** la cantidad se actualiza en tiempo real
2. **Dado** que reduzco la cantidad a 0 **cuando** confirmo **entonces** el item se elimina del carrito
3. **Dado** que modifico cantidades **cuando** veo el resumen **entonces** el subtotal y total se recalculan correctamente
4. **Dado** que intento cantidad negativa **cuando** presiono - en cantidad 1 **entonces** no permite ir por debajo de 1

Etiquetas:

HU-006

Como cliente, **quiero** eliminar items de mi carrito, **para** quitar productos que ya no deseo comprar.

Criterios de Aceptación:

1. **Dado** que estoy en `/carrito` **cuando** hago clic en "Eliminar" en un item **entonces** el item desaparece de la lista
2. **Dado** que elimino todos los items **cuando** el carrito queda vacío **entonces** veo estado vacío con botón para ir al catálogo
3. **Dado** que elimino un item **cuando** veo el resumen **entonces** el total se actualiza restando el subtotal del item eliminado
4. **Dado** que uso "Vaciar carrito" **cuando** confirmo **entonces** se eliminan todos los items de una vez

Etiquetas:

HU-007

Como cliente, **quiero** que mi carrito persista entre sesiones y se fusione al iniciar sesión, **para** no perder mi selección al autenticarme.

Criterios de Aceptación:

1. **Dado** que agrego items sin estar logueado **cuando** cierro y abro el navegador **entonces** el carrito sigue

intacto (localStorage)

2. **Dado** que tengo items en localStorage **cuando** inicio sesión con Clerk **entonces** el carrito se fusiona con el del usuario autenticado
3. **Dado** que tenía items en ambas fuentes **cuando** hay duplicados **entonces** se suman las cantidades por pastelId
4. **Dado** que me deslogueo **cuando** reviso el carrito **entonces** veo el estado localStorage previo

Etiquetas:

HU-008 [MCP]

Como cliente autenticado, **quiero** ver mi carrito actual con totales, **para** revisar mi pedido antes de proceder al checkout.

Criterios de Aceptación:

1. **Dado** que estoy autenticado **cuando** accedo a `/carrito` **entonces** veo la lista de items con imagen, nombre, precio, cantidad, subtotal
2. **Dado** que tengo items en el carrito **cuando** veo el resumen **entonces** veo subtotal, envío (calculado en checkout), y total
3. **Dado** que el carrito está vacío **cuando** accedo a `/carrito` **entonces** veo mensaje "Tu Carrito está Vacío" con enlace al catálogo
4. **Dado** que modifico items **cuando** veo el resumen **entonces** los totales se actualizan en tiempo real

Etiquetas: [MCP] | Tool: `get\_cart` | Auth: `User JWT`

HU-009

Como cliente, **quiero** seleccionar método de entrega (domicilio o tienda) en checkout, **para** definir cómo recibiré mi pedido.

Criterios de Aceptación:

1. **Dado** que estoy en `/checkout` **cuando** cargo la página **entonces** veo dos opciones: "Envío a Domicilio" y "Retiro en Tienda"
2. **Dado** que selecciono "Envío a Domicilio" **cuando** continúo **entonces** aparecen campos obligatorios: dirección, referencias
3. **Dado** que selecciono "Retiro en Tienda" **cuando** continúo **entonces** no se requieren campos de dirección
4. **Dado** que cambio de opción **cuando** vuelvo a la anterior **entonces** se muestran/ocultan los campos correspondientes

Etiquetas:

HU-010

Como cliente, **quiero** ingresar mis datos de contacto (email, teléfono) en checkout, **para** que el pedido se asocie a mi cuenta y pueda recibir confirmaciones.

Criterios de Aceptación:

1. **Dado** que estoy en `/checkout` **cuando** cargo la página **entonces** veo campos obligatorios: email (validado formato), teléfono
3. **Dado** que ingreso email inválido **cuando** intento enviar **entonces** veo error de validación en cliente

4. **Dado** que dejen campos vacíos **cuando** intento enviar **entonces** el formulario no se envía y muestra errores requeridos

Etiquetas:

HU-011

Como cliente, **quiero** enviar mi pedido al backend para crear una sesión de Stripe Checkout, **para** proceder al pago seguro.

Criterios de Aceptación:

1. **Dado** que completo el formulario de checkout **cuando** presiono "Pagar con Stripe" **entonces** se envía POST a `/api/pedidos` con token Clerk, items, método entrega, email, teléfono, dirección
2. **Dado** que el backend valida y crea el pedido **cuando** responde **entonces** retorna `checkoutUrl` de Stripe y `pedidoId`
3. **Dado** que recibo la checkoutUrl **cuando** el frontend redirige **entonces** navego a Stripe Checkout externo
4. **Dado** que hay error de autenticación **cuando** el token expira **entonces** redirige a `/checkout/error?reason=no\_token`

Etiquetas:

HU-012

Como cliente, **quiero** ser redirigido a Stripe Checkout para completar el pago, **para** pagar de forma segura sin que la aplicación maneje datos de tarjeta.

Criterios de Aceptación:

1. **Dado** que el backend crea la sesión Stripe **cuando** el frontend redirige **entonces** voy a `checkout.stripe.com` con los line items correctos (precio * 1.07 IVA incluido)
2. **Dado** que estoy en Stripe **cuando** completo el pago **entonces** Stripe redirige a `/checkout/exito?session\_id={id}&order\_id={id}`
3. **Dado** que cancelo en Stripe **cuando** regreso **entonces** voy a `/checkout/error` con mensaje de cancelado
4. **Dado** que la sesión expira **cuando** intento pagar **entonces** Stripe muestra error de sesión inválida

Etiquetas:

HU-013

Como cliente, **quiero** ver la página de éxito después de pagar, **para** confirmar que mi pedido fue procesado.

Criterios de Aceptación:

1. **Dado** que Stripe redirige a `/checkout/exito` **cuando** cargo la página **entonces** veo mensaje de confirmación con número de pedido
2. **Dado** que la URL tiene `session\_id` y `order\_id` **cuando** muestro la página **entonces** se muestran ambos IDs para referencia
3. **Dado** que el pago se confirmó **cuando** voy a `/pedidos` **entonces** veo el nuevo pedido en estado PAGADO
4. **Dado** que recargo la página de éxito **cuando** no hay parámetros **entonces** muestro mensaje genérico de éxito

Etiquetas:

HU-014

Como cliente, **quiero** ver la página de error si el pago falla o cancelo, **para** entender qué pasó y reintentar.

Criterios de Aceptación:

1. **Dado** que Stripe redirige a `/checkout/error` **cuando** cargo la página **entonces** veo mensaje de error con posible razón (cancelado, error, no_token)
2. **Dado** que el error es `reason=no\_token` **cuando** muestro la página **entonces** veo botón para ir a login y reintentar
3. **Dado** que el error es genérico **cuando** muestro la página **entonces** veo enlace para volver al carrito e intentar de nuevo
4. **Dado** que hay error de red **cuando** el webhook no llega **entonces** el pedido queda en PENDIENTE y puedo reintentar desde `/pedidos`

Etiquetas:

HU-015

Como sistema, **quiero** recibir y validar webhooks de Stripe (checkout.session.completed), **para** confirmar pagos de forma segura y asíncrona.

Criterios de Aceptación:

1. **Dado** que Stripe envía POST a `/api/webhook/stripe` **cuando** llega el evento **entonces** valido la firma HMAC con `STRIPE\_WEBHOOK\_SECRET`
2. **Dado** que la firma es inválida **cuando** proceso el webhook **entonces** respondo 400 "Firma inválida" y no proceso el evento
3. **Dado** que el evento es `checkout.session.completed` **cuando** valido **entonces** busco el pedido por `stripeSessionId` y actualizo estado a PAGADO
4. **Dado** que el pedido se confirma **cuando** actualizo **entonces** envío email de factura via Resend al cliente

Etiquetas: [MCP] | Tool: `handle\_stripe\_webhook` | Auth: `Webhook Secret (HMAC SHA256)`

HU-016 [MCP]

Como cliente autenticado, **quiero** ver mi historial de pedidos con filtros, **para** rastrear el estado de mis compras.

Criterios de Aceptación:

1. **Dado** que estoy autenticado **cuando** accedo a `/pedidos` **entonces** veo lista de mis pedidos con paginación (backend filtra por clerkUserId)
2. **Dado** que tengo pedidos **cuando** uso el filtro de estado **entonces** veo solo pedidos con ese estado (PENDIENTE, PAGADO, PREPARANDO, LISTO, EN_CAMINO, ENTREGADO, CANCELADO)
3. **Dado** que veo un pedido **cuando** inspecciono la tarjeta **entonces** veo: ID corto, fecha, método entrega, items con precios snapshot, total, badge de estado con color
4. **Dado** que no estoy autenticado **cuando** accedo a `/pedidos` **entonces** veo mensaje para iniciar sesión con enlace a `/sign-in`

Etiquetas: [MCP] | Tool: `list\_orders` | Auth: `User JWT`

HU-017 [MCP]

Como cliente autenticado, **quiero** ver el detalle completo de un pedido mío, **para** revisar items, totales, dirección y timeline de estados.

Criterios de Aceptación:

1. **Dado** que estoy en `/pedidos` **cuando** hago clic en un pedido **entonces** veo detalle con: items (nombre, cantidad, precio snapshot, subtotal), total, método entrega, dirección si domicilio, teléfono
2. **Dado** que el pedido tiene timeline **cuando** veo el detalle **entonces** veo el estado actual con badge de color y fecha de creación
3. **Dado** que soy admin **cuando** veo cualquier pedido **entonces** tengo acceso completo (verificado en backend con header x-user-role)
4. **Dado** que intento ver pedido ajeno **cuando** no soy admin **entonces** recibo 403 "Acceso denegado"

Etiquetas: [MCP] | Tool: `get\_order` | Auth: `User JWT`

HU-018

Como cliente, **quiero** acceder a la página de recetas personalizadas, **para** solicitar un pastel a medida (actualmente placeholder).

Criterios de Aceptación:

1. **Dado** que estoy autenticado **cuando** accedo a `/recetas` **entonces** veo página "Próximamente" con mensaje de que la funcionalidad está en desarrollo
2. **Dado** que no estoy autenticado **cuando** accedo a `/recetas` **entonces** el middleware Clerk me redirige a `/sign-in`
3. **Dado** que la página carga **cuando** inspecciono **entonces** hay enlaces funcionales a `/catalogo` y `/pedidos`
4. **Dado** que el backend tiene endpoints **cuando** reviso `routes/recetas.ts` **entonces** veo GET, POST, PUT, PUT/cotizar implementados pero sin UI frontend

Etiquetas:

HU-019 [MCP]

Como cliente autenticado, **quiero** enviar una receta personalizada con archivo y nota, **para** que el admin la revise y cotice (backend listo, frontend pendiente).

Criterios de Aceptación:

1. **Dado** que el backend tiene POST `/api/recetas` **cuando** envío `nota`, `personas`, `archivoUrl` **entonces** crea receta en estado PENDIENTE con clerkUserId
2. **Dado** que subo archivo a `/api/upload/receta` **cuando** recibo respuesta **entonces** obtengo `archivoUrl` de Cloudinary para usar en creación
3. **Dado** que creo la receta **cuando** recibo respuesta **entonces** veo la receta con ID, estado PENDIENTE, cotización null
4. **Dado** que el frontend no tiene UI **cuando** intento usar la funcionalidad **entonces** no hay formulario para

enviar recetas (pendiente implementar)

Etiquetas: [MCP] | Tool: `submit\_recipe` | Auth: `User JWT`

HU-020 [MCP]

Como cliente autenticado, **quiero** ver mis recetas enviadas con sus estados y cotizaciones, **para** dar seguimiento a mis solicitudes (backend listo, frontend pendiente).

Criterios de Aceptación:

1. **Dado** que el backend tiene GET `/api/recetas` **cuando** consulto con mi token **entonces** recibo lista de mis recetas filtradas por clerkUserId
2. **Dado** que veo mis recetas **cuando** inspecciono cada una **entonces** veo: ID, nota (truncada), fecha, estado (PENDIENTE/REVISANDO/COTIZADA/ACEPTADA/RECHAZADA), cotización si existe
3. **Dado** que una receta está COTIZADA **cuando** la veo **entonces** veo el precio cotizado y puedo aceptarla
4. **Dado** que el frontend no tiene UI **cuando** intento ver mis recetas **entonces** no hay página para listarlas (pendiente implementar)

Etiquetas: [MCP] | Tool: `list\_my\_recipes` | Auth: `User JWT`

HU-021 [MCP]

Como cliente autenticado, **quiero** aceptar una cotización de receta y proceder al pago, **para** convertir mi receta en un pedido real (backend listo, frontend pendiente).

Criterios de Aceptación:

1. **Dado** que mi receta está en estado COTIZADA **cuando** decido aceptarla **entonces** el backend crea un pedido vinculado a la receta con el precio cotizado
2. **Dado** que se crea el pedido **cuando** el backend responde **entonces** retorna `checkoutUrl` de Stripe para pagar la receta
3. **Dado** que pago en Stripe **cuando** el webhook confirma **entonces** la receta pasa a ACEPTADA y el pedido a PAGADO
4. **Dado** que el frontend no tiene UI **cuando** intento aceptar **entonces** no hay botón ni flujo para hacerlo (pendiente implementar)

Etiquetas: [MCP] | Tool: `accept\_quote` | Auth: `User JWT`

HU-022 [MCP]

Como administrador, **quiero** ver un dashboard con KPIs del negocio, **para** monitorear ventas, pedidos y recetas pendientes.

Criterios de Aceptación:

1. **Dado** que soy admin autenticado **cuandocuando accedo a `/admin` entonces**** veo 4 tarjetas: Pedidos Hoy, Recetas Pendientes, Productos, Ingresos Mes
2. **Dado** que cargo el dashboard **cuando** el backend responde **entonces** `GET /api/admin/dashboard` retorna stats calculadas en tiempo real

3. **Dado** que los datos cambian **cuando** recargo **entonces** veo valores actualizados (pedidos hoy, ingresos mes, etc.)
4. **Dado** que no soy admin **cuando** intento acceder **entonces** el middleware Clerk + backend verifyAdmin me bloquean (403)

Etiquetas: [MCP] | Tool: `admin\_get\_dashboard\_stats` | Auth: `Admin JWT`

HU-023

Como administrador, **quiero** ver la lista completa de pasteles en el panel admin, **para** gestionar el catálogo.

Criterios de Aceptación:

1. **Dado** que soy admin **cuando** accedo a `/admin/pasteles` **entonces** veo tabla con: Nombre, Categoría, Precio, Estado (Activo/Agotado), Acciones
2. **Dado** que hay pasteles **cuando** cargo la página **entonces** `GET /api/admin/pasteles` retorna todos con paginación y búsqueda opcional
3. **Dado** que veo la tabla **cuando** inspecciono filas **entonces** el badge de estado muestra verde "Activo" o rojo "Agotado" según `disponible`
4. **Dado** que no soy admin **cuando** intento acceder **entonces** Clerk middleware + backend verifyAdmin me redirigen/bloquean

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` | Auth: `Admin JWT`

HU-024 [MCP]

Como administrador, **quiero** crear un nuevo pastel en el catálogo, **para** ofrecer nuevos productos a los clientes.

Criterios de Aceptación:

1. **Dado** que estoy en `/admin/pasteles` **cuando** hago clic en "+ Nuevo Pastel" **entonces** se abre modal con formulario: nombre, descripción, precio, categoría, imagen
2. **Dado** que completo el formulario **cuando** subo imagen **entonces** `POST /api/upload` sube a Cloudinary y retorna URL
3. **Dado** que tengo imagen y datos **cuando** envío **entonces** `POST /api/admin/pasteles` crea el pastel en MongoDB y actualiza la tabla
4. **Dado** que falda nombre o precio ≤ 0 **cuando** envío **entonces** validación Zod rechaza con error 400

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` (POST) | Auth: `Admin JWT`

HU-025 [MCP]

Como administrador, **quiero** editar un pastel existente, **para** actualizar su información, precio o disponibilidad.

Criterios de Aceptación:

1. **Dado** que estoy en `/admin/pasteles` **cuando** hago clic en editar (icono lápiz) **entonces** se abre modal precargado con datos del pastel
2. **Dado** que modifico campos **cuando** guardo **entonces** `PUT /api/admin/pasteles/:id` actualiza en MongoDB y

refresca la tabla

3. **Dado** que cambio imagen **cuando** subo nueva **entonces** reemplaza la URL en Cloudinary y actualiza el registro

4. **Dado** que toggleo disponibilidad **cuando** guardo **entonces** el badge en tabla cambia entre "Activo"/"Agotado" en tiempo real

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` (PUT) | Auth: `Admin JWT`

HU-026 [MCP]

Como administrador, **quiero** eliminar un pastel del catálogo, **para** retirar productos que ya no se venden.

Criterios de Aceptación:

1. **Dado** que estoy en `/admin/pasteles` **cuando** hago clic en eliminar (icono basura) **entonces** pide confirmación antes de borrar

2. **Dado** que confirmo **cuando** envío **entonces** `DELETE /api/admin/pasteles/:id` elimina de MongoDB y quita de la tabla

3. **Dado** que el pastel tiene pedidos previos **cuando** elimino **entonces** se elimina (los pedidos guardan precio snapshot, no ref rota)

4. **Dado** que no soy admin **cuando** intento **entonces** 403 por verifyAdmin

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` (DELETE) | Auth: `Admin JWT`

HU-027

Como administrador, **quiero** ver todos los pedidos del sistema con filtros, **para** gestionar el flujo de preparación y entrega.

Criterios de Aceptación:

1. **Dado** que soy admin **cuando** accedo a `/admin/pedidos` **entonces** veo tabla: ID, Fecha, Total, Estado, Acciones (dropdown estado)

2. **Dado** que uso filtro de estado **cuando** selecciono **entonces** `GET /api/admin/pedidos?status=X` filtra solo ese estado

3. **Dado** que veo la tabla **cuando** inspecciono **entonces** cada fila tiene dropdown con estados válidos: PENDIENTE, CONFIRMADO, PAGADO, ENTREGADO, CANCELADO

4. **Dado** que no soy admin **cuando** intento **entonces** bloqueado por middleware + verifyAdmin

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` (GET /api/admin/pedidos) | Auth: `Admin JWT`

HU-028 [MCP]

Como administrador, **quiero** actualizar el estado de un pedido, **para** reflejar su progreso (preparando, listo, en camino, entregado).

Criterios de Aceptación:

1. **Dado** que estoy en `/admin/pedidos` **cuando** cambio el dropdown de estado **entonces** `PUT /api/admin/pedidos/:id/status` actualiza en MongoDB
2. **Dado** que actualizo estado **cuando** recibo respuesta **entonces** la tabla refleja el nuevo estado sin recargar (optimistic update)
3. **Dado** que el estado sigue flujo válido **cuando** actualizo **entonces** PENDIENTE → PAGADO → PREPARANDO → LISTO → EN_CAMINO → ENTREGADO (CANCELADO desde cualquier estado)
4. **Dado** que intento estado inválido **cuando** envío **entonces** Zod valida y rechaza con 400

Etiquetas: [MCP] | Tool: `admin\_update\_order\_status` | Auth: `Admin JWT`

HU-029

Como administrador, **quiero** ver todas las recetas personalizadas enviadas por clientes, **para** revisarlas y cotizarlas.

Criterios de Aceptación:

1. **Dado** que soy admin **cuando** accedo a `/admin/recetas` **entonces** veo tabla: ID, Nota (truncada), Fecha, Estado, Cotización, Acciones
2. **Dado** que cargo la página **cuando** `GET /api/admin/recetas` responde **entonces** lista todas con estados: PENDIENTE, REVISANDO, COTIZADA, ACEPTADA, RECHAZADA
3. **Dado** que veo una receta PENDIENTE **cuando** la inspecciono **entonces** el badge muestra amarillo "PENDIENTE" y botón "Cotizar"
4. **Dado** que no soy admin **cuando** intento **entonces** bloqueado

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` (GET /api/admin/recetas) | Auth: `Admin JWT`

HU-030 [MCP]

Como administrador, **quiero** cotizar una receta personalizada ingresando un precio, **para** que el cliente pueda aceptarla y pagar.

Criterios de Aceptación:

1. **Dado** que estoy en `/admin/recetas` **cuando** hago clic en "Cotizar" en una receta PENDIENTE **entonces** se abre prompt para ingresar precio
2. **Dado** que ingreso precio válido > 0 **cuando** confirmo **entonces** `PUT /api/admin/recetas/:id` con `{cotizacion: X, estado: "COTIZADA"}` actualiza la receta
3. **Dado** que la receta se cotiza **cuando** el cliente la ve **entonces** su UI (pendiente) mostraría el precio y botón "Aceptar y Pagar"
4. **Dado** que ingreso precio inválido **cuando** envío **entonces** validación rechaza y pide precio correcto

Etiquetas: [MCP] | Tool: `admin\_quote\_recipe` | Auth: `Admin JWT`

HU-031

Como usuario, **quiero** registrarme e iniciar sesión con Clerk (Google SSO, Magic Link, OTP), **para** acceder a funcionalidades protegidas.

Criterios de Aceptación:

1. **Dado** que voy a `/sign-in` o `/sign-up` **cuando** cargo la página **entonces** veo componentes Clerk integrados con opciones: Google, email (Magic Link/OTP)
2. **Dado** que me registro con Google **cuando** autorizo **entonces** Clerk crea usuario y redirige a callback configurado
3. **Dado** que uso Magic Link **cuando** ingreso email **entonces** recibo email con enlace de acceso único
4. **Dado** que uso OTP **cuando** ingreso email **entonces** recibo código de 6 dígitos por email para autenticar

Etiquetas:

HU-032

Como usuario autenticado, **quiero** que las rutas protegidas (/carrito, /pedidos, /recetas, /admin) requieran sesión, **para** que solo usuarios logueados accedan.

Criterios de Aceptación:

1. **Dado** que no estoy logueado **cuando** intento acceder a `/carrito` **entonces** Clerk middleware me redirige a `/sign-in?redirect\_url=/carrito`
2. **Dado** que no estoy logueado **cuando** intento acceder a `/pedidos` **entonces** redirige a sign-in con redirect
3. **Dado** que no estoy logueado **cuando** intento acceder a `/admin/\*` **entonces** redirige a sign-in
4. **Dado** que estoy logueado pero no soy admin **cuando** intento `/admin` **entonces** backend verifyAdmin retorna 403

Etiquetas:

HU-033

Como backend, **quiero** verificar tokens Clerk en cada request autenticado, **para** identificar al usuario y validar su sesión.

Criterios de Aceptación:

1. **Dado** que llega request con `Authorization: Bearer <token>` **cuando** `authMiddleware` procesa **entonces** `verifyToken` usa `@clerk/backend` con `CLERK\_SECRET\_KEY`
2. **Dado** que el token es válido **cuando** verifica **entonces** retorna `userId` (sub del JWT) para usar en controllers
3. **Dado** que el token es inválido/expirado **cuando** verifica **entonces** retorna null y endpoints responden 401
4. **Dado** que no hay header Authorization **cuando** procesa **entonces** retorna null inmediatamente

Etiquetas:

HU-034

Como backend, **quiero** verificar si un usuario autenticado tiene rol admin/superadmin, **para** autorizar acceso a endpoints administrativos.

Criterios de Aceptación:

1. **Dado** que ``verifyAdmin`` recibe `userId` válido **cuando** consulta Clerk ``getUser`` **entonces** lee ``publicMetadata.role``
2. **Dado** que role es "admin" o "superadmin" **cuando** verifica **entonces** retorna ``{userId, role}`` para permitir acceso
3. **Dado** que role es "user" o no existe **cuando** verifica **entonces** retorna ``{userId, role: 'user'}`` y endpoints admin responden 403
4. **Dado** que Clerk falla **cuando** hay error de red **entonces** asume role 'user' por seguridad (fail-closed)

Etiquetas:

HU-035

Como desarrollador, **quiero** ejecutar un script de seed que cree 19 pasteles con imágenes en Cloudinary, **para** poblar la base de datos con datos de prueba realistas.

Criterios de Aceptación:

1. **Dado** que ejecuto ``bun scripts/seed.ts`` **cuando** conecta a MongoDB **entonces** limpia colección Pastel y crea 19 registros
2. **Dado** que procesa cada pastel **cuando** sube imagen local a Cloudinary **entonces** reemplaza filename por URL CDN en el registro
3. **Dado** que termina **cuando** cuenta documentos **entonces** reporta "Total de pasteles: 19" con categorías: chocolate(5), vainilla(5), frutas(5), especial(4)
4. **Dado** que falla una imagen **cuando** procesa **entonces** loggea error pero continúa con los demás

Etiquetas:

HU-036 [MCP]

Como sistema de monitoreo, **quiero** consultar el endpoint de health check, **para** verificar que el backend y sus dependencias están operativos.

Criterios de Aceptación:

1. **Dado** que hago GET a ``/health`` **cuando** el servidor está arriba **entonces** responde 200 con ``{status: "ok", timestamp: "ISO8601"}``
2. **Dado** que MongoDB está caído **cuando** el servidor inicia **entonces** el proceso falla en ``connectDB()`` y no llega a escuchar puerto
3. **Dado** que el endpoint es público **cuando** consulto sin auth **entonces** responde sin requerir autenticación
4. **Dado** que uso load balancer **cuando** configuro health check **entonces** este endpoint sirve para readiness/liveness probes

Etiquetas: [MCP] | Tool: ``health_check`` | Auth: 'Público (sin auth)'

HU-037 [MCP]

Como administrador, **quiero** subir imágenes de pasteles a Cloudinary via API, **para** usar URLs CDN en el catálogo sin almacenar archivos localmente.

Criterios de Aceptación:

1. **Dado** que hago POST a `/api/upload` con `multipart/form-data` field "file" **cuando** soy admin **entonces** sube buffer a Cloudinary folder `dolce-atelier/catalogo`
2. **Dado** que la subida exitos **cuando** responde **entonces** retorna `{url: "https://res.cloudinary.com/.../image.jpg"}`
3. **Dado** que no envío archivo **cuando** hago request **entonces** responde 400 "No se proporcionó archivo"
4. **Dado** que no soy admin **cuando** intento **entonces** verifyAdmin bloquea con 401/403

Etiquetas: [MCP] | Tool: `admin\_list\_cakes` (POST /api/upload) | Auth: `Admin JWT`

HU-038

Como cliente, **quiero** subir archivos de receta personalizada a Cloudinary, **para** adjuntar mi receta en formato PDF/imagen al enviarla.

Criterios de Aceptación:

1. **Dado** que hago POST a `/api/upload/receta` con field "archivo" y opcional "nombre" **cuando** soy usuario autenticado **entonces** sube a Cloudinary
2. **Dado** que la subida exitos **cuando** responde **entonces** retorna `{url: "https://res.cloudinary.com/.../receta.pdf"}` para usar en creación de receta
3. **Dado** que el archivo excede límite **cuando** subo **entonces** Cloudinary rechaza y retorna error 500
4. **Dado** que no envío archivo **cuando** hago request **entonces** responde 400

Etiquetas:

HU-039

Como cliente, **quiero** recibir email de factura después de pagar, **para** tener comprobante de mi compra con detalle de items.

Criterios de Aceptación:

1. **Dado** que el webhook Stripe confirma pago **cuando** `confirmarPagoConEmail` actualiza pedido **entonces** llama `enviarFacturaPedido` via Resend
2. **Dado** que se envía email **cuando** llega al cliente **entonces** incluye: nombre cliente, pedidoId, total, items (nombre, cantidad, precio snapshot), método entrega, dirección si domicilio
3. **Dado** que el email se envía **cuando** Resend responde **entonces** loggea éxito en backend
4. **Dado** que falla el email **cuando** Resend error **entonces** loggea error pero no revierte la confirmación de pago

Etiquetas:

HU-040

Como desarrollador, **quiero** ejecutar un script que levante backend y frontend simultáneamente, **para** desarrollar full-stack con un solo comando.

Criterios de Aceptación:

1. **Dado** que ejecuto ``bun scripts/dev.ts`` **cuando** corre **entonces** spawna ``bun run src/server.ts`` (backend puerto 3001) y ``npm run dev`` en frontend (puerto 3000)
2. **Dado** que ambos procesos corren **cuando** hay output **entonces** muestra logs prefijados [BACKEND] y [FRONTEND] en la misma consola
3. **Dado** que presiono Ctrl+C **cuando** detengo **entonces** mata ambos procesos hijos limpiamente
4. **Dado** que hay error en uno **cuando** falla **entonces** el script reporta y permite debug independiente

Etiquetas:

Historias Production Ready

HU-041

Como arquitecto de seguridad, **quiero** implementar rate limiting en todos los endpoints de API, **para** prevenir abuso, brute force y DoS.

Criterios de Aceptación:

1. **Dado** que configuro rate limiter (token bucket: 100 req/min por IP, 1000 req/min por API Key) **cuando** excede límite **entonces** responde 429 con ``Retry-After`` header
2. **Dado** que endpoints auth (login, register, password reset) **cuando** configuro **entonces** límite estricto: 5 intentos/15min por IP/email
3. **Dado** que webhook Stripe **cuando** configuro **entonces** exento de rate limit (Stripe maneja su propio retry)
4. **Dado** que rate limit aplica **cuando** responde 429 **entonces** incluye headers: ``X-RateLimit-Limit``, ``X-RateLimit-Remaining``, ``X-RateLimit-Reset``

Etiquetas: [MCP] | Tool: ``health_check`` (indirecto) | Auth: ``API Key + User JWT`` (rate limit por ambos)

HU-042

Como arquitecto de seguridad, **quiero** implementar rotación automática de JWT y refresh tokens, **para** reducir ventana de exposición si un token es comprometido.

Criterios de Aceptación:

1. **Dado** que Clerk emite access tokens **cuando** configuro **entonces** access token TTL = 15min, refresh token TTL = 7 días
2. **Dado** que access token expira **cuando** frontend detecta 401 **entonces** usa refresh token automáticamente para obtener nuevo access token
3. **Dado** que refresh token se usa **cuando** rota **entonces** invalida el anterior (rotación estricta) y emite nuevo par
4. **Dado** que detecto uso de refresh token revocado **cuando** ocurre **entonces** revoco toda la sesión y fuerzo re-login

Etiquetas:**HU-043**

Como arquitecto de seguridad, **quiero** implementar Content Security Policy (CSP) estricta en frontend, **para** mitigar XSS e inyección de código.

Criterios de Aceptación:

1. **Dado** que configuro `next.config.js` con headers CSP **cuando** cargo páginas **entonces** `script-src 'self' 'unsafe-inline' https://clerk.example.com https://js.stripe.com`
2. **Dado** que CSP está activo **cuando** intento inyectar script externo no permitido **entonces** navegador bloquea y reporta a `report-uri`
3. **Dado** que Clerk y Stripe necesitan dominios **cuando** configuro **entonces** `connect-src` incluye `api.clerk.com`, `api.stripe.com`, `clerk.example.com`
4. **Dado** que desarrollo **cuando** `NODE\_ENV=development` **entonces** CSP en modo `report-only` para no bloquear hot-reload

Etiquetas:**HU-044**

Como arquitecto de seguridad, **quiero** implementar auditoría de accesos y cambios críticos, **para** tener trazabilidad completa para compliance y forense.

Criterios de Aceptación:

1. **Dado** que ocurre evento sensible (login, cambio rol, creación pedido, webhook, admin CRUD) **cuando** se procesa **entonces** loggea: timestamp, userId, action, resourceId, ip, userAgent, resultado
2. **Dado** que los logs de auditoría **cuando** se almacenan **entonces** van a colección MongoDB `audit\_logs` con TTL 2 años, índices por userId, action, timestamp
3. **Dado** que admin consulta auditoría **cuando** usa endpoint **entonces** `GET /api/admin/audit` con filtros: userId, action, dateRange, paginación
4. **Dado** que hay intento de acceso no autorizado **cuando** 403/401 **entonces** también se audita con `resultado: "DENIED"``

Etiquetas: [MCP] | Tool: `admin\_search\_users` (extensión) | Auth: `SuperAdmin JWT`

HU-045

Como arquitecto de seguridad, **quiero** implementar escaneo automático de secretos en CI/CD, **para** evitar que credenciales lleguen a repositorio.

Criterios de Aceptación:

1. **Dado** que configuro `git-leaks` o `truffleHog` en GitHub Actions **cuando** hay push/PR **entonces** escanea todo el historial y diff por patrones: API keys, JWT, passwords, connection strings
2. **Dado** que detecta secreto **cuando** falla el job **entonces** bloquea merge y alerta en Slack/email con archivo y línea
3. **Dado** que secretos legítimos en `.env.example` **cuando** configuro **entonces** usa allowlist para no false-positives
4. **Dado** que rota secretos **cuando** detecta fuga **entonces** documenta procedimiento de rotación en runbook

Etiquetas:

HU-046

Como ingeniero de pagos, **quiero** implementar claves de idempotencia en creación de pedidos y webhooks, **para** evitar cobros duplicados por reintentos de red.

Criterios de Aceptación:

1. **Dado** que cliente envía POST `/api/pedidos` **cuando** incluye header `Idempotency-Key: <uuid>` **entonces** backend verifica si ya procesó esa key en últimas 24h
2. **Dado** que key existe **cuando** reintenta **entonces** retorna respuesta original guardada (pedidoId, checkoutUrl) sin crear nuevo pedido ni sesión Stripe
3. **Dado** que webhook Stripe llega duplicado **cuando** procesa **entonces** usa `event.id` como idempotency key natural y evita doble confirmación
4. **Dado** que almacena idempotency keys **cuando** usa Redis/Mongo **entonces** TTL 24h, índice único en key, cleanup job diario

Etiquetas: [MCP] | Tool: `create\_order` | Auth: `User JWT`

HU-047

Como ingeniero de pagos, **quiero** implementar lógica de reintentos exponenciales para webhooks Stripe fallidos, **para** garantizar eventual consistency ante fallos transitorios.

Criterios de Aceptación:

1. **Dado** que webhook falla (timeout, 5xx, error DB) **cuando** Stripe reintenta **entonces** respeta backoff exponencial: 1min, 5min, 15min, 1h, 3h, 6h, 12h, 24h (máx 72h)
2. **Dado** que webhook falla persistentemente **cuando** supera reintentos **entonces** mueve a Dead Letter Queue (colección `webhook\_dlq`) con payload completo y error
3. **Dado** que DLQ tiene eventos **cuando** admin revisa **entonces** `GET /api/admin/webhooks/dlq` lista con opción de re-procesar manualmente

4. **Dado** que re-procesa manual **cuando** confirma **entonces** elimina de DLQ y procesa normalmente

Etiquetas: [MCP] | Tool: `handle\_stripe\_webhook` | Auth: `Webhook Secret + Admin JWT (para DLQ)`

HU-048

Como ingeniero de pagos, **quiero** implementar reembolsos parciales y totales desde panel admin, **para** manejar devoluciones y cancelaciones post-pago.

Criterios de Aceptación:

1. **Dado** que pedido está en estado PAGADO/ENTREGADO **cuando** admin inicia reembolso **entonces** `POST /api/admin/pedidos/:id/refund` con `{amount?: number, reason: string}`
2. **Dado** que amount omitido **cuando** procesa **entonces** reembolso total via Stripe
`refunds.create({payment\_intent: ..., amount: total\*100})`
3. **Dado** que amount especificado **cuando** procesa **entonces** reembolso parcial (validar amount <= total pagado - ya reembolsado)
4. **Dado** que reembolso exitoso **cuando** Stripe confirma **entonces** actualiza pedido: `estado: CANCELADO` (total) o añade campo `refundedAmount`, `refundReason`, `refundedAt`

Etiquetas: [MCP] | Tool: `admin\_update\_order\_status` (extensión) | Auth: `Admin JWT`

HU-049

Como ingeniero de pagos, **quiero** documentar y evidenciar cumplimiento PCI DSS SAQ-A, **para** operar legalmente procesando tarjetas via Stripe.

Criterios de Aceptación:

1. **Dado** que usamos Stripe Checkout (redirect) **cuando** evaluamos **entonces** calificamos para SAQ-A (no tocamos PAN, solo token)
2. **Dado** que SAQ-A requiere **cuando** documentamos **entonces**: política seguridad, diagramas flujo datos, inventario sistemas, scans ASV trimestrales, penetration test anual
3. **Dado** que evidencia **cuando** auditor pide **entonces** generamos: Attestation of Compliance (AoC), Responsibility Matrix, network diagrams
4. **Dado** que renovación anual **cuando** vence **entonces** recordatorio automático 60 días antes con checklist actualizado

Etiquetas:

HU-050

Como ingeniero de observabilidad, **quiero** implementar logging estructurado JSON con niveles y correlación, **para** debuggear eficiente y análisis automatizado.

Criterios de Aceptación:

1. **Dado** que configuro `pino` en backend **cuando** loggea **entonces** output JSON: `{level, time, pid, hostname, reqId, userId, action, message, meta}`
2. **Dado** que request entra **cuando** middleware **entonces** genera `reqId` (UUID v4) y propaga en headers `X-Request-ID` a servicios downstream
3. **Dado** que loggea en services/controllers **cuando** usa logger **entonces** incluye contexto: `userId`, `pedidoId`, `recetaId`, `durationMs`
4. **Dado** que logs van a stdout **cuando** Docker/K8s **entonces** recolector (Fluent Bit/Vector) envía a Loki/Elasticsearch con labels: app, env, level

Etiquetas: [MCP] | Tool: `health\_check` (logs incluidos) | Auth: `Público (logs no sensibles)`

HU-051

Como ingeniero de observabilidad, **quiero** implementar métricas Prometheus (RED + USE) en backend, **para** alertar y dashboarding de salud del sistema.

Criterios de Aceptación:

1. **Dado** que expongo `/metrics` **cuando** Prometheus scrapea **entonces** ve: RED (rate, errors, duration) por endpoint: `http\_requests\_total{method,path,status}`, `http\_request\_duration\_seconds{method,path,quantile}`
2. **Dado** que métricas de negocio **cuando** ocurren eventos **entonces** custom metrics: `orders\_created\_total`, `orders\_paid\_total`, `recipes\_submitted\_total`, `recipes\_quoted\_total`, `webhook\_processed\_total{status}`
3. **Dado** que métricas de recursos **cuando** Node.js **entonces** USE: `process\_cpu\_seconds\_total`, `process\_resident\_memory\_bytes`, `nodejs\_eventloop\_lag\_seconds`, `mongodb\_connections\_active`
4. **Dado** que alertas **cuando** configuro **entonces** reglas: `rate(http\_requests\_total{status=~"5.."}[5m]) > 0.05`, `http\_request\_duration\_seconds{p99} > 2`, `orders\_paid\_total[1h] == 0`

Etiquetas: [MCP] | Tool: `health\_check` (métricas incluidas) | Auth: `Público (/metrics)`

HU-052

Como ingeniero de observabilidad, **quiero** implementar distributed tracing con OpenTelemetry, **para** rastrear latencia y errores across frontend, backend, DB, Stripe, Cloudinary.

Criterios de Aceptación:

1. **Dado** que configuro OTEL SDK en Next.js y Elysia **cuando** request inicia **entonces** crea trace con `traceId`, `spanId`, propaga via `traceparent` header
2. **Dado** que spans se crean **cuando** instrumenta **entonces** spans automáticos: HTTP client (fetch), MongoDB (mongoose), Stripe SDK, Cloudinary SDK, Clerk SDK
3. **Dado** que spans tienen atributos **cuando** enriquezco **entonces** `http.method`, `http.route`, `http.status\_code`, `db.operation`, `db.collection`, `user.id`, `pedido.id`
4. **Dado** que exporta a collector **cuando** deploya **entonces** OTEL Collector → Jaeger/Tempo/Grafana Cloud con retención 7d, sampling 10% (100% errors)

Etiquetas: [MCP] | Tool: `health\_check` (tracing health) | Auth: `Público`

HU-053

Como ingeniero de observabilidad, **quiero** configurar alertas accionables en PagerDuty/Slack, **para** responder a incidentes antes que clientes lo noten.

Criterios de Aceptación:

1. **Dado** que defino alertas críticas **cuando** configuro **entonces**: P1 - API down (health check fail 3x), DB down, Stripe webhook failing 5min, error rate > 5%
2. **Dado** que defino alertas de advertencia **cuando** configuro **entonces**: P2 - p95 latency > 2s, queue depth webhook DLQ > 10, disk > 80%, memory > 85%
3. **Dado** que alerta dispara **cuando** notifica **entonces** incluye: runbook link, dashboard link, logs query, metric query, on-call escalation
4. **Dado** que alerta resuelta **cuando** auto-resuelve **entonces** cierra incidente y notifica "resolved" con duration

Etiquetas:

HU-054

Como ingeniero de confiabilidad, **quiero** implementar health checks profundos (liveness + readiness), **para** que orquestador (K8s/Railway) gestione tráfico y reinicios correctamente.

Criterios de Aceptación:

1. **Dado** que GET `/health/live` **cuando** proceso vivo **entonces** 200 OK inmediato (sin deps check) - para liveness probe
2. **Dado** que GET `/health/ready` **cuando** listo para tráfico **entonces** verifica: MongoDB ping < 100ms, Stripe API reachable, Cloudinary ping, Clerk JWKS fetch < 500ms
3. **Dado** que alguna dep falla **cuando** readiness **entonces** 503 con body: `{checks: [{name: "mongodb", status: "fail", latencyMs: 5000}]}` y Kubernetes quita pod de service
4. **Dado** que startup probe **cuando** configuro **entonces** `/health/startup` igual a readiness pero timeout 60s para migraciones/seed

Etiquetas: [MCP] | Tool: `health\_check` | Auth: `Público`

HU-055

Como ingeniero de frontend, **quiero** implementar Error Boundaries en React para capturar errores de render, **para** mostrar fallback graceful en lugar de pantalla blanca.

Criterios de Aceptación:

1. **Dado** que envuelvo rutas en `` **cuando** componente lanza error **entonces** captura y renderiza fallback UI con: mensaje amigable, botón "Recargar", botón "Reportar", requestId
2. **Dado** que error boundary captura **cuando** loggea **entonces** envía a Sentry/LogRocket con: error stack, component stack, userId, url, userAgent
3. **Dado** que error en layout **cuando** root boundary **entonces** muestra página de error completa manteniendo header/footer navegables
4. **Dado** que error en async (useEffect) **cuando** configuro **entonces** `onError` en QueryClient / SWR también reporta

Etiquetas:

HU-056

Como ingeniero de backend, **quiero** implementar manejo global de errores con códigos consistentes, **para** que clientes (y MCP) manejen errores programáticamente.

Criterios de Aceptación:

1. **Dado** que ocurre error no capturado **cuando** `onError` en Elysia **entonces** responde JSON: `{error: {code: "INTERNAL\_ERROR", message: "...", requestId: "...", timestamp: "..."}}`
2. **Dado** que error de validación Zod **cuando** parse falla **entonces** 400 con `{code: "VALIDATION\_ERROR", details: [{path, message, code}]}`
3. **Dado** que error de auth **cuando** 401/403 **entonces** `{code: "UNAUTHORIZED" | "FORBIDDEN", message: "..."}`
4. **Dado** que error de not found **cuando** 404 **entonces** `{code: "NOT\_FOUND", resource: "pastel|pedido|receta", id: "..."}`

Etiquetas: [MCP] | Tool: `health\_check` (error format) | Auth: `API Key + User JWT`

HU-057

Como ingeniero de backend, **quiero** implementar Dead Letter Queue para webhooks y jobs fallidos, **para** no perder eventos y permitir reprocesamiento manual.

Criterios de Aceptación:

1. **Dado** que webhook/job falla tras max reintentos **cuando** mueve a DLQ **entonces** inserta en `webhook\_dlq`: `{eventId, eventType, payload, error, attempts, lastAttemptAt, createdAt}`
2. **Dado** que DLQ crece **cuando** admin monitorea **entonces** `GET /api/admin/dlq` con paginación, filtros: eventType, dateRange, status (pending/retrying/resolved)
3. **Dado** que admin reprocesa **cuando** POST `/api/admin/dlq/:id/retry` **entonces** re-ejecuta handler original, si éxito marca `resolved`, si falla incrementa attempts
4. **Dado** que DLQ antigua **cuando** job cleanup **entonces** archiva > 90 días a cold storage (S3/Blob) y purga de Mongo

Etiquetas: [MCP] | Tool: `handle\_stripe\_webhook` (DLQ access) | Auth: `Admin JWT`

HU-058

Como ingeniero de calidad, **quiero** implementar tests unitarios con Vitest (cobertura $\geq 80\%$), **para** validar lógica de negocio aislada de dependencias externas.

Criterios de Aceptación:

1. **Dado** que configuro Vitest + @vitest/coverage-v8 **cuando** ejecuto `bun test` **entonces** corre tests en `backend/src/\*\*/\*.test.ts` y `frontend/\*\*/\*.test.tsx`
2. **Dado** que testeo services **cuando** mockeo **entonces** mock: MongoDB (mongoose-mock), Stripe

(stripe-mock), Cloudinary, Resend, Clerk

3. **Dado** que testeo `pedidoService.crear` **cuando** valido **entonces** casos: éxito, pastel no encontrado, stock insuficiente, validación Zod falla, Stripe error

4. **Dado** que cobertura **cuando** reporte **entonces** lines $\geq 80\%$, branches $\geq 70\%$, functions $\geq 80\%$, statements $\geq 80\%$

Etiquetas:

HU-059

Como ingeniero de calidad, **quiero** implementar tests de integración con Testcontainers (MongoDB real), **para** validar integración real con base de datos.

Criterios de Aceptación:

1. **Dado** que configuro Testcontainers en Vitest **cuando** inicio test **entonces** levanta MongoDB 7.0 en container efímero, aplica migraciones/índices

2. **Dado** que testeo routes **cuando** uso supertest **entonces** hace requests HTTP reales a Elysia app contra MongoDB real

3. **Dado** que testeo flujo pedido **cuando** integro **entonces**: crear pastel → crear pedido → webhook mock → verificar estado PAGADO → email enviado

4. **Dado** que limpieza **cuando** test termina **entonces** Testcontainers mata container, sin residuos entre tests

Etiquetas:

HU-060

Como ingeniero de calidad, **quiero** implementar tests E2E con Playwright (Chromium, Firefox, WebKit), **para** validar flujos críticos usuario real en navegadores reales.

Criterios de Aceptación:

1. **Dado** que configuro Playwright **cuando** ejecuto `npx playwright test` **entonces** corre specs en `e2e/\*.spec.ts` contra `http://localhost:3000` (preview deploy)

2. **Dado** que testeo flujo compra **cuando** spec **entonces**: login Clerk (test user) → catálogo → add to cart → checkout → Stripe test mode → success → verify order in /pedidos

3. **Dado** que testeo admin **cuando** spec **entonces**: login admin → dashboard → crear pastel → editar → eliminar → verificar catálogo público

4. **Dado** que CI **cuando** corre **entonces** artefactos: screenshots on failure, trace.zip, video, HTML report; timeout 60s por test

Etiquetas:

HU-061

Como ingeniero de calidad, **quiero** implementar contract testing con Pact (consumer-driven), **para** garantizar

compatibilidad frontend-backend y MCP tools.

Criterios de Aceptación:

1. **Dado** que defino contratos **cuando** frontend/MCP consumen **entonces** generan pacts: `frontend-pact.json`, `mcp-tools-pact.json` con interacciones esperadas
2. **Dado** que backend provee **cuando** CI corre **entonces** `pact-verifier` valida que provider (Elysia) cumple todos los contratos consumidos
3. **Dado** que contrato rompe **cuando** verifica **entonces** falla build con diff claro: `Expected status 200 but got 404`, `Missing field 'checkoutUrl'`
4. **Dado** que versionado **cuando** publico **entonces** pacts en Pact Broker con tags: `main`, `prod`, `staging`; can-i-deploy gate en deploy

Etiquetas: [MCP] | Tool: `health\_check` (contract validation) | Auth: `API Key + User JWT`

HU-062

Como ingeniero de calidad, **quiero** implementar mutation testing con Stryker, **para** medir calidad real de tests (no solo cobertura).

Criterios de Aceptación:

1. **Dado** que configuro Stryker para Vitest **cuando** ejecuto `npx stryker run` **entonces** muta operadores: aritméticos, lógicos, condicionales, literales en `backend/src/services/\*\*/\*.ts`
2. **Dado** que mutantes generados **cuando** corren tests **entonces** reporte: mutation score $\geq 70\%$, killed $\geq 80\%$, survived $< 20\%$, timeout $< 5\%$
3. **Dado** que mutantes sobreviven **cuando** analizo **entonces** identifico gaps: tests que no validan边界条件, falta asserts en catch blocks
4. **Dado** que CI **cuando** gate **entonces** mutation score $< 70\%$ bloquea merge; baseline por archivo para no degradar

Etiquetas:

HU-063

Como ingeniero de plataforma, **quiero** implementar pipeline CI/CD en GitHub Actions (matrix, caching, gates), **para** deploy automatizado y confiable a staging/producción.

Criterios de Aceptación:

1. **Dado** que push a main **cuando** workflow `ci.yml` **entonces** jobs paralelos: lint (eslint, prettier), typecheck (tsc --noEmit), unit, integration, contract
2. **Dado** que tests pasan **cuando** build **entonces** `bun build` backend → Docker image multi-arch (amd64/arm64) → push a GHCR con tags: `sha`, `main`, `v\*`
3. **Dado** que frontend **cuando** build **entonces** `next build` → static export / standalone → Docker image → push GHCR
4. **Dado** que deploy staging **cuando** main pasa **entonces** auto-deploy a Railway (backend) y Vercel (frontend) via CLI/token; smoke tests post-deploy

Etiquetas:

HU-064

Como ingeniero de plataforma, **quiero** implementar preview deployments por PR, **para** validar cambios visuales y funcionales antes de merge.

Criterios de Aceptación:

1. **Dado** que se abre PR **cuando** GitHub Actions **entonces** despliega a: `pr-<number>.staging.dolceatelier.com` (Vercel preview) + Railway preview environment
2. **Dado** que preview está listo **cuando** comenta en PR **entonces** botón "Ver Preview", URL backend preview, resumen de checks
3. **Dado** que preview expira **cuando** PR cierra/mergea **entonces** limpieza automática de recursos (Vercel auto, Railway via webhook)
4. **Dado** que E2E contra preview **cuando** configuro **entonces** Playwright corre subset crítico (login, catálogo, checkout) contra URL preview

Etiquetas:

HU-065

Como ingeniero de plataforma, **quiero** implementar blue-green deployment en Railway, **para** zero-downtime deploy con rollback instantáneo.

Criterios de Aceptación:

1. **Dado** que nueva versión lista **cuando** deploy **entonces** Railway crea nuevo servicio `backend-green` junto a `backend-blue` (actual)
2. **Dado** que health checks pasan en green **cuando** switch **entonces** actualiza DNS/load balancer a green; blue queda en standby 10min
3. **Dado** que hay error en green **cuando** detecta **entonces** rollback automático a blue (< 30s) y alerta P1
4. **Dado** que rollback manual **cuando** admin **entonces** `railway service switch blue` via CLI o dashboard; audit log registra quién y por qué

Etiquetas:

HU-066

Como ingeniero de plataforma, **quiero** implementar rollback automatizado por métricas de salud, **para** revertir deployments defectuosos sin intervención humana.

Criterios de Aceptación:

1. **Dado** que deploy completo **cuando** monitorea **entonces** observa 10min: error rate, p95 latency, business metrics (orders/hour)
2. **Dado** que métricas cruzan umbral **cuando** evalúa **entonces** si `error\_rate > 5%` OR `p95 > 3s` OR `orders\_paid[10min] == 0` → trigger rollback
3. **Dado** que rollback ejecuta **cuando** revierte **entonces** Railway switch a versión previa; notifica

Slack/PagerDuty con: versión anterior, métricas trigger, link dashboard

4. **Dado** que post-rollback **cuando** estabiliza **entonces** mantiene versión estable; crea incidente Jira/Linear para post-mortem; bloquea re-deploy hasta approval

Etiquetas:

HU-067

Como arquitecto de seguridad, **quiero** implementar RBAC granular (resource:action) en backend, **para** principio de menor privilegio en APIs y MCP tools.

Criterios de Aceptación:

1. **Dado** que defino permisos **cuando** model **entonces** recursos: `pastel`, `pedido`, `receta`, `usuario`, `audit`, `webhook`; acciones: `create`, `read`, `update`, `delete`, `list`, `export`, `quote`
2. **Dado** que roles base **cuando** asigno **entonces**: `cliente` (pastel:read,list; pedido:create,read,list own; receta:create,read,list own), `admin` (all except usuario:delete, audit:export), `superadmin` (all)
3. **Dado** que middleware `requirePermission(resource, action)` **cuando** protege endpoints **entonces** verifica `user.permissions.includes(`\${resource}:\${action}`) o role wildcard
4. **Dado** que MCP tools **cuando** exponen **entonces** cada tool declara `requiredPermission` y gateway valida antes de ejecutar handler

Etiquetas: [MCP] | Tool: `admin\_search\_users` (gestión permisos) | Auth: `SuperAdmin JWT`

HU-068

Como administrador, **quiero** UI en `/admin/usuarios` para asignar/revocar roles, **para** gestionar permisos sin editar Clerk dashboard manualmente.

Criterios de Aceptación:

1. **Dado** que soy superadmin **cuando** accedo a `/admin/usuarios` **entonces** veo tabla: Email, Nombre, Rol actual (badge), Último login, Acciones
2. **Dado** que hago clic en editar rol **cuando** modal **entonces** dropdown con roles: `cliente`, `admin`, `superadmin`; botón guardar llama `PATCH /api/admin/usuarios/:id/role`
3. **Dado** que guardo **cuando** backend **entonces** actualiza `publicMetadata.role` en Clerk via `@clerk/backend` y invalida sesión (force refresh token)
4. **Dado** que intento degradar superadmin **cuando** soy el último **entonces** bloquea con error "No puede degradar al último superadmin"

Etiquetas: [MCP] | Tool: `admin\_search\_users` (PATCH role) | Auth: `SuperAdmin JWT`

HU-069

Como administrador de soporte, **quiero** funcionalidad de impersonación (login as user), **para** debugar problemas de clientes sin pedir credenciales.

Criterios de Aceptación:

1. **Dado** que soy admin **cuando** en `/admin/usuarios` hago "Impersonar" **entonces** backend genera short-lived token (5min) con `act: {sub: adminId}` claim
2. **Dado** que token impersonado **cuando** uso **entonces** frontend muestra banner "Modo impersonación: usuario@email.com" con botón "Salir"
3. **Dado** que acciones en modo impersonación **cuando** audit **entonces** logs marcan `impersonatedBy: adminId` en todos los eventos
4. **Dado** que seguridad **cuando** configuro **entonces** solo superadmin puede impersonar; no puede impersonar a otro admin/superadmin; max 1 sesión simultánea

Etiquetas: [MCP] | Tool: `admin\_search\_users` (impersonate) | Auth: `SuperAdmin JWT`

HU-070

Como arquitecto de seguridad, **quiero** implementar ABAC (Attribute-Based Access Control) para recetas, **para** reglas dinámicas: cliente ve solo propias, admin ve todas, solo admin cotiza, cliente solo acepta propias cotizadas.

Criterios de Aceptación:

1. **Dado** que policy engine (OPA/Cedar o custom) **cuando** evalúa `receta:read` **entonces** permite si: `user.id == receta.clerkUserId` OR `user.role in [admin, superadmin]`
2. **Dado** que `receta:quote` **cuando** evalúa **entonces** permite solo si `user.role in [admin, superadmin]` AND `receta.estado in [PENDIENTE, REVISANDO]`
3. **Dado** que `receta:accept` **cuando** evalúa **entonces** permite si `user.id == receta.clerkUserId` AND `receta.estado == COTIZADA` AND `receta.cotizacion > 0`
4. **Dado** que MCP tools **cuando** exponen **entonces** cada tool valida policy engine antes de ejecutar; decision loggea para auditoría

Etiquetas: [MCP] | Tool: `list\_my\_recipes`, `accept\_quote`, `admin\_quote\_recipe` | Auth: `User JWT / Admin JWT`

HU-071

Como arquitecto de seguridad, **quiero** implementar protección anti-fuerza bruta en endpoints de autenticación, **para** prevenir credential stuffing y account takeover.

Criterios de Aceptación:

1. **Dado** que endpoint Clerk (login, register, password reset, MFA) **cuando** configuro **entonces** rate limit: 5 intentos/15min por IP + 3 intentos/hora por email
2. **Dado** que excede límite **cuando** bloquea **entonces** 429 con `Retry-After: 900`; muestra CAPTCHA (hCaptcha/turnstile) tras 3 fallos
3. **Dado** que IP en lista negra **cuando** detecta **entonces** bloqueo permanente hasta revisión manual; integra con AbuseIPDB/Cloudflare threat intel
4. **Dado** que usuario legítimo bloqueado **cuando** soporta **entonces** admin puede desbloquear IP/email en `/admin/security/blocklist` con auditoría

Etiquetas:

HU-072

Como arquitecto de plataforma, **quiero** implementar rate limiting por API Key en gateway MCP, **para** cuota justa entre consumidores (Claude Code, Codex, dashboards internos).

Criterios de Aceptación:

1. **Dado** que emito API Keys MCP **cuando** configuro **entonces** planes: `free` (100 req/min, 10k/day), `pro` (1000 req/min, 1M/day), `enterprise` (custom)
2. **Dado** que request llega **cuando** gateway **entonces** valida API Key, extrae plan, aplica token bucket (Redis: `rl:mcp:{apiKey}` con refill rate = plan.rpm/60)
3. **Dado** que excede cuota **cuando** responde **entonces** 429 con `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, `Retry-After`
4. **Dado** que analytics **cuando** admin **entonces** `GET /api/admin/mcp/usage` con breakdown por API Key, endpoint, status, latency p50/p95/p99

Etiquetas: [MCP] | Tool: `health\_check` (rate limit status) | Auth: `API Key`

HU-073

Como ingeniero de confiabilidad, **quiero** configurar Point-in-Time Recovery (PITR) en MongoDB Atlas, **para** recuperar a cualquier segundo en ventana de 7 días.

Criterios de Aceptación:

1. **Dado** que cluster Atlas M10+ **cuando** habilito PITR **entonces** snapshots continuos cada 10s; retención 7 días; costo ~10% storage
2. **Dado** que necesito recuperar **cuando** ejecuto **entonces** `atlas clusters restore` especificando `timestamp: "2024-01-15T10:30:00Z"` → nuevo cluster con datos a ese punto
3. **Dado** que testeo recovery **cuando** trimestral **entonces** drill documentado: restaura a cluster temporal, valida integridad (conteos, índices, sample queries), destruye
4. **Dado** que RPO/RPO **cuando** documentado **entonces** RPO < 10s (PITR granularity), RTO < 30min (restore + DNS switch); SLA interno 99.9%

Etiquetas:

HU-074

Como ingeniero de confiabilidad, **quiero** script de recovery de datos semilla (pasteles, usuarios admin), **para** reconstruir estado mínimo funcional tras desastre total.

Criterios de Aceptación:

1. **Dado** que cluster limpio **cuando** ejecuto `bun scripts/disaster-recovery.ts` **entonces**: crea índices, inserta 19 pasteles base (desde `seed.ts` pero idempotente), crea 1 superadmin (configurable via env)
2. **Dado** que imágenes Cloudinary **cuando** recovery **entonces** verifica que URLs en seed existen en

Cloudinary; si no, re-sube desde `frontend/public/images/catalogo/` (requiere credenciales)

3. **Dado** que usuarios Clerk **cuando** recovery **entonces** no restaura usuarios (Clerk es source of truth); solo asegura que superadmin existe y tiene `publicMetadata.role=superadmin`

4. **Dado** que validación **cuando** termina **entonces** health check completo: `/health`, `/api/pasteles` (19 items), `/api/admin/stats` (superadmin auth)

Etiquetas:

HU-075

Como arquitecto de confiabilidad, **quiero** documentar RTO/RPO y runbooks de disaster recovery, **para** que equipo responda consistentemente ante incidentes mayores.

Criterios de Aceptación:

1. **Dado** que documento **cuando** publico **entonces** `RUNBOOK\_DISASTER\_RECOVERY.md` con: arquitectura, dependencias críticas, RTO/RPO por componente, contactos on-call, procedimientos paso a paso
2. **Dado** que componentes **cuando** clasifico **entonces**: MongoDB (RTO 30min, RPO 10s), Backend (RTO 5min, RPO 0 - stateless), Frontend (RTO 2min, RPO 0), Stripe/Clerk/Cloudinary (externos - status page)
3. **Dado** que escenarios **cuando** cubro **entonces**: región caída, data corruption, ransomware, credential leak, DNS hijack, cascade failure
4. **Dado** que drills **cuando** programo **entonces** table-top exercise mensual + full failover trimestral; métricas: tiempo a detectar, tiempo a recuperar, datos perdidos

Etiquetas:

HU-076

Como ingeniero de frontend, **quiero** alcanzar conformidad WCAG 2.1 AA en todo el frontend, **para** accesibilidad legal e inclusiva.

Criterios de Aceptación:

1. **Dado** que audito con axe-core + Lighthouse **cuando** CI **entonces** 0 violations AA; score accesibilidad ≥ 95
2. **Dado** que navego solo teclado **cuando** Tab/Shift+Tab/Enter/Esc **entonces** todos los elementos interactivos enfocables, orden lógico, focus visible (outline 2px solid #e11d48)
3. **Dado** que uso screen reader (NVDA/VoiceOver) **cuando** pruebo **entonces**: landmarks (main, nav, aside), headings jerárquicos (h1→h2→h3), labels en inputs, alt text en imágenes, anuncios de estado (aria-live)
4. **Dado** que contraste **cuando** verifico **entonces** ratio $\geq 4.5:1$ texto normal, $\geq 3:1$ large text/UI components; no solo color para transmitir info (badges con texto + color)

Etiquetas:

HU-077

Como ingeniero de frontend, **quiero** implementar navegación por teclado completa y skip links, **para** usuarios

que no usan mouse.

Criterios de Aceptación:

1. **Dado** que cargo página **cuando** presiono Tab **entonces** primer foco: "Saltar al contenido principal" (skip link visible al enfocar)
2. **Dado** que en catálogo **cuando** navego **entonces** Tarjetas de pastel: Enter = ver detalle; en detalle: Enter en "Agregar" = add to cart; carrito: flechas para cantidad, Enter para eliminar
3. **Dado** que en modales admin **cuando** abro **entonces** focus trap (Tab cicla dentro), Esc cierra, focus regresa a trigger al cerrar
4. **Dado** que en checkout **cuando** formulario **entonces** orden lógico: email → teléfono → método entrega → dirección (condicional) → pagar; validación anuncia errores con `aria-describedby`

Etiquetas:

HU-078

Como ingeniero de frontend, **quiero** implementar labels ARIA, live regions y semántica HTML5, **para** compatibilidad total con screen readers.

Criterios de Aceptación:

1. **Dado** que inputs **cuando** renderizo **entonces** cada `` tiene `` visible o `aria-label`/`aria-labelledby`; `aria-invalid`, `aria-describedby` para errores
2. **Dado** que estados dinámicos **cuando** cambian **entonces** carrito total: `Total: \$XXX`; notificaciones toast: `role="status" aria-live="polite"`
3. **Dado** que badges de estado **cuando** renderizo **entonces** `PAGADO` (no solo color)
4. **Dado** que tablas admin **cuando** renderizo **entonces** `

Etiquetas:

HU-079

Como ingeniero de frontend, **quiero** implementar internacionalización (i18n) ES/EN con next-intl, **para** soportar mercado mexicano y futuro expansión US.

Criterios de Aceptación:

1. **Dado** que configuro `next-intl` **cuando** routing **entonces** URLs: `/es/catalogo`, `/en/catalog`; middleware detecta `Accept-Language` y redirige a locale por defecto (es-MX)
2. **Dado** que diccionarios **cuando** estructuro **entonces** `messages/es.json`, `messages/en.json` con namespaces: `common`, `catalog`, `cart`, `checkout`, `orders`, `recipes`, `admin`
3. **Dado** que formateo **cuando** uso **entonces** `useFormatter()` para: moneda (MXN/USD con `Intl.NumberFormat`), fechas (`es-MX`: dd/mm/yyyy, `en-US`: mm/dd/yyyy), números, plurales
4. **Dado** que SSR **cuando** renderiza **entonces** `getLocale()` en server components; `Link` genera hrefs con locale; sitemap incluye alternates hreflang

Etiquetas:

HU-080

Como ingeniero de frontend, **quiero** preparar CSS RTL-ready para futuro soporte árabe/hebreo, **para** arquitectura i18n completa sin refactor mayor.

Criterios de Aceptación:

1. **Dado** que uso CSS logical properties **cuando** escribo estilos **entonces** ``margin-inline-start`` en lugar de ``margin-left``, ``padding-inline-end`` vs ``padding-right``, ``border-inline-start``
2. **Dado** que flexbox/grid **cuando** direccional **entonces** ``flex-direction: row`` → ``flex-direction: row-reverse`` via ``[dir="rtl"] &``; ``justify-content: flex-start`` → ``flex-end`` logical
3. **Dado** que iconos direccionales **cuando** condicionales **entonces** chevron right/left: ``transform: scaleX(-1)`` en ``[dir="rtl"]`` o SVGs separados
4. **Dado** que testing **cuando** verifico **entonces** Storybook/Chromatic con ``dir="rtl"`` en viewport; visual regression tests comparan LTR vs RTL

Etiquetas:

Catálogo de Herramientas MCP - Dolce Atelier

Definición MCP

Model Context Protocol (MCP) para Dolce Atelier expone herramientas tipadas sobre la API REST existente, permitiendo a agentes IA (Claude Code, Codex, Cursor) operar el e-commerce programáticamente bajo autenticación ``API Key + User JWT``. Cada tool mapea 1:1 a un caso de uso atómico del dominio.

Autenticación MCP

Todas las herramientas MCP requieren headers:

X-API-Key: <mcp_api_key> # Identifica al cliente MCP (plan, cuota)
Authorization: Bearer <user_jwt> # Identifica al usuario final (Clerk)

Excepciones:

- ``health_check``: No requiere auth

- `handle\_stripe\_webhook`: Usa `stripe-signature` HMAC, no JWT

Planes API Key:

Plan	RPM	Requests/día	Costo
free	100	10,000	Gratis
pro	1,000	1,000,000	\$99/mes
enterprise	Custom	Custom	Contactar

Tool: `search\_cakes`

Historia: HU-001 [MCP]

Descripción: Búsqueda tipada de catálogo con filtros compuestos y paginación.

Input Schema:json

```
{
  "type": "object",
  "properties": {
    "categoria": {
      "type": "string",
      "enum": ["chocolate", "vainilla", "frutas", "especial"],
      "description": "Filtrar por categoría de pastel"
    },
    "precioMin": {
      "type": "number",
      "minimum": 0,
      "description": "Precio mínimo en MXN"
    },
    "precioMax": {
      "type": "number",
      "minimum": 0,
      "description": "Precio máximo en MXN"
    },
    "disponible": {
      "type": "boolean",
      "default": true,
      "description": "Solo pasteles disponibles"
    },
    "q": {
      "type": "string",
      "description": "Búsqueda por texto libre (nombre, descripción)"
    }
  }
}
```

```

    "page": {
      "type": "number",
      "default": 1,
      "minimum": 1
    },
    "limit": {
      "type": "number",
      "default": 12,
      "minimum": 1,
      "maximum": 50
    }
  },
  "additionalProperties": false
}

```

Output Schema:

json

```

{
  "type": "object",
  "properties": {
    "pasteles": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "_id": { "type": "string" },
          "nombre": { "type": "string" },
          "descripcion": { "type": "string" },
          "precio": { "type": "number" },
          "imagen": { "type": "string", "format": "uri" },
          "categoria": { "type": "string" },
          "disponible": { "type": "boolean" }
        }
      }
    },
    "total": { "type": "number" },
    "page": { "type": "number" },
    "totalPages": { "type": "number" }
  }
}

```

Endpoint: `GET /api/pasteles?categoria=chocolate&precioMin=400&precioMax=600&page=1&limit=12`

Auth: `API Key + User JWT`

Rate Limit: 60 req/min por API Key

Idempotencia: GET (seguro)

Errores:

Código	Significado
--------	-------------

- 401 JWT inválido o expirado
- 401 API Key inválida o inactiva
- 429 Rate limit excedido (ver [Retry-After](#))
- 500 Error interno (log: [requestId](#))

Tool: `get\_cake\_detail`

Historia: HU-002 [MCP]

Descripción: Obtener detalle completo de un pastel por ID.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "pastelId": {
      "type": "string",
      "description": "ID MongoDB del pastel"
    }
  },
  "required": ["pastelId"]
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "nombre": { "type": "string" },
    "descripcion": { "type": "string" },
    "precio": { "type": "number" },
    "imagen": { "type": "string", "format": "uri" },
    "categoria": { "type": "string" },
    "disponible": { "type": "boolean" },
    "createdAt": { "type": "string", "format": "date-time" },
    "updatedAt": { "type": "string", "format": "date-time" }
  }
}
```

Endpoint: `GET /api/pasteles/:id`

Auth: `API Key + User JWT`

Rate Limit: 60 req/min por API Key

Idempotencia: GET (seguro)

Errores:

Código	Significado
404	Pastel no encontrado

Tool: `add\_to\_cart`

Historia: HU-004 [MCP]

Descripción: Agregar un pastel al carrito de compras del usuario autenticado.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "pastelId": {
      "type": "string",
      "description": "ID MongoDB del pastel"
    },
    "cantidad": {
      "type": "integer",
      "minimum": 1,
      "default": 1,
      "description": "Cantidad a agregar"
    }
  },
  "required": ["pastelId"]
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "success": { "type": "boolean" },
    "pastel": {
      "type": "object",
      "properties": {
        "_id": { "type": "string" },

```

```

        "nombre": { "type": "string" },
        "precio": { "type": "number" },
        "imagen": { "type": "string" }
    }
},
"cantidad": { "type": "integer" },
"carrito": {
    "type": "object",
    "properties": {
        "items": {
            "type": "array",
            "items": {
                "type": "object",
                "properties": {
                    "pastelId": { "type": "string" },
                    "nombre": { "type": "string" },
                    "precio": { "type": "number" },
                    "imagen": { "type": "string" },
                    "cantidad": { "type": "integer" }
                }
            }
        }
    },
    "total": { "type": "number" },
    "totalItems": { "type": "integer" }
}
}
}
}

```

Endpoint: `POST /api/cart/items` (nuevo endpoint MCP) o mapeo a Zustand store

Auth: `User JWT`

Rate Limit: 120 req/min por usuario

Idempotencia: Sí (agregar mismo pastel incrementa cantidad, no duplica línea)

Errores:

Código	Significado
404	Pastel no encontrado
409	Pastel no disponible (disponible=false)
422	Cantidad inválida (≤ 0 o $>$ stock máximo)

Tool: `get\_cart`

Historia: HU-008 [MCP]

Descripción: Obtener el carrito actual con items, totales y validación de stock.

Input Schema:

```
json
{
  "type": "object",
  "properties": {},
  "additionalProperties": false
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "pastelId": { "type": "string" },
          "nombre": { "type": "string" },
          "precio": { "type": "number" },
          "imagen": { "type": "string" },
          "cantidad": { "type": "integer" },
          "subtotal": { "type": "number" },
          "disponible": { "type": "boolean" }
        }
      }
    },
    "subtotal": { "type": "number" },
    "totalItems": { "type": "integer" },
    "itemsNoDisponibles": {
      "type": "array",
      "description": "Items que ya no están disponibles"
    }
  }
}
```

Endpoint: `GET /api/cart` (nuevo endpoint MCP) o lectura desde Zustand store

Auth: `User JWT`

****Rate Limit:** 60 req/min por usuario

Idempotencia: GET (seguro)

Errores:

Código	Significado
401	No autenticado

Tool: `create\_order`

Historia: HU-014 [MCP]

Descripción: Crear pedido desde el carrito y generar sesión de Stripe Checkout.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "email": {
      "type": "string",
      "format": "email",
      "description": "Email del cliente para confirmación"
    },
    "items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "pastelId": { "type": "string" },
          "cantidad": { "type": "integer", "minimum": 1 }
        },
        "required": ["pastelId", "cantidad"]
      },
      "minItems": 1
    },
    "metodoEntrega": {
      "type": "string",
      "enum": ["DOMICILIO", "TIENDA"],
      "description": "Método de entrega del pedido"
    },
    "direccionEnvio": {
      "type": "string",
      "description": "Dirección completa (requerido si metodoEntrega=DOMICILIO)"
    },
    "telefono": {
      "type": "string",
      "description": "Teléfono de contacto"
    }
  },
}
```

```

    "required": ["email", "items", "metodoEntrega"]
  }

```

Output Schema:

```

json
{
  "type": "object",
  "properties": {
    "pedido": {
      "type": "object",
      "properties": {
        "_id": { "type": "string" },
        "estado": { "type": "string", "enum": ["PENDIENTE"] },
        "total": { "type": "number" },
        "items": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "pastelId": { "type": "string" },
              "nombre": { "type": "string" },
              "precioSnapshot": { "type": "number" },
              "cantidad": { "type": "integer" }
            }
          }
        },
        "metodoEntrega": { "type": "string" },
        "createdAt": { "type": "string", "format": "date-time" }
      }
    },
    "checkoutUrl": {
      "type": "string",
      "format": "uri",
      "description": "URL de Stripe Checkout para completar el pago"
    }
  }
}

```

Endpoint: `POST /api/pedidos`

Auth: `User JWT`

****Rate Limit:** 30 req/min por usuario (prevenir creación masiva)

Idempotencia: Requiere `Idempotency-Key` header para evitar duplicados

Errores:

Código	Significado
400	Validación Zod falla (email inválido, items vacíos, etc.)

401	No autenticado
404	Pastel no encontrado en DB
409	Pastel no disponible
422	Dirección requerida para DOMICILIO
429	Rate limit
500	Error creando sesión Stripe

Tool: `list\_orders`

Historia: HU-016 [MCP]

Descripción: Listar pedidos del usuario autenticado (o todos si es admin) con filtros.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "estado": {
      "type": "string",
      "enum": ["PENDIENTE", "PAGADO", "PREPARANDO", "LISTO", "EN_CAMINO", "ENTREGADO", "CANCELADO"],
      "description": "Filtrar por estado del pedido"
    },
    "page": {
      "type": "number",
      "default": 1
    },
    "limit": {
      "type": "number",
      "default": 20,
      "maximum": 100
    }
  },
  "additionalProperties": false
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
```

```

"pedidos": {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "_id": { "type": "string" },
      "estado": { "type": "string" },
      "total": { "type": "number" },
      "items": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "nombre": { "type": "string" },
            "precioSnapshot": { "type": "number" },
            "cantidad": { "type": "integer" }
          }
        }
      },
      "metodoEntrega": { "type": "string" },
      "createdAt": { "type": "string", "format": "date-time" }
    }
  },
  "total": { "type": "number" },
  "page": { "type": "number" },
  "totalPages": { "type": "number" }
}
}

```

Endpoint: `GET /api/pedidos?estado=PAGADO`

Auth: `User JWT` (clientes ven solo los suyos; admins con header `X-User-Role: admin` ven todos)

Rate Limit: 60 req/min por usuario

Idempotencia: GET (seguro)

Errores:

Código	Significado
401	No autenticado
403	Intentando acceder a pedidos de otro usuario sin rol admin

Tool: `get\_order`

Historia: HU-017 [MCP]

Descripción: Obtener detalle completo de un pedido específico.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "pedidoId": {
      "type": "string",
      "description": "ID MongoDB del pedido"
    }
  },
  "required": ["pedidoId"]
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "estado": { "type": "string" },
    "total": { "type": "number" },
    "email": { "type": "string" },
    "items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "pastelId": { "type": "string" },
          "nombre": { "type": "string" },
          "precioSnapshot": { "type": "number" },
          "cantidad": { "type": "integer" }
        }
      }
    },
    "metodoEntrega": { "type": "string" },
    "direccionEnvio": { "type": "string" },
    "telefono": { "type": "string" },
    "stripeSessionId": { "type": "string" },
    "createdAt": { "type": "string", "format": "date-time" },
    "updatedAt": { "type": "string", "format": "date-time" }
  }
}
```

****Endpoint:** `GET /api/pedidos/:id`

Auth: `User JWT` (dueño del pedido o admin)

Rate Limit: 60 req/min por usuario

Idempotencia: GET (seguro)

Errores:

Código	Significado
401	No autenticado
403	No es dueño ni admin
404	Pedido no encontrado

Tool: `submit\_recipe`

Historia: HU-019 [MCP]

Descripción: Enviar una receta personalizada con archivo y notas.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "nota": {
      "type": "string",
      "minLength": 10,
      "description": "Descripción detallada del pastel deseado"
    },
    "personas": {
      "type": "integer",
      "default": 10,
      "minimum": 1,
      "maximum": 200,
      "description": "Número de personas que servirá el pastel"
    },
    "archivoUrl": {
      "type": "string",
      "format": "uri",
      "description": "URL del archivo subido vía /api/upload/receta"
    }
  },
  "required": ["nota"]
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "receta": {
      "type": "object",
      "properties": {
        "_id": { "type": "string" },
        "nota": { "type": "string" },
        "personas": { "type": "integer" },
        "archivoUrl": { "type": "string" },
        "estado": { "type": "string", "enum": ["PENDIENTE"] },
        "cotizacion": { "type": "null" },
        "createdAt": { "type": "string", "format": "date-time" }
      }
    }
  }
}
```

Endpoint: `POST /api/recetas`

Auth: `User JWT`

Rate Limit: 10 req/min por usuario (prevenir spam)

Idempotencia: No (cada envío crea nueva receta)

Errores:

Código	Significado
400	Validación falla (nota muy corta, personas inválida)
401	No autenticado
413	Archivo excede límite de tamaño (10MB)
415	Tipo de archivo no soportado (solo PDF, JPG, PNG)

Tool: `list\_my\_recipes`

Historia: HU-020 [MCP]

Descripción: Listar recetas personalizadas del usuario autenticado.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "estado": {
      "type": "string",
      "enum": ["PENDIENTE", "REVISANDO", "COTIZADA", "ACEPTADA", "RECHAZADA"],
      "description": "Filtrar por estado de la receta"
    }
  },
  "additionalProperties": false
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "recetas": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "_id": { "type": "string" },
          "nota": { "type": "string" },
          "personas": { "type": "integer" },
          "archivoUrl": { "type": "string" },
          "estado": { "type": "string" },
          "cotizacion": { "type": ["number", "null"] },
          "createdAt": { "type": "string", "format": "date-time" }
        }
      }
    }
  }
}
```

Endpoint: `GET /api/recetas?estado=COTIZADA`

Auth: `User JWT` (clientes ven solo las suyas; admins ven todas)

Rate Limit: 60 req/min por usuario

Idempotencia: GET (seguro)

Errores:

Código	Significado
--------	-------------

401	No autenticado
-----	----------------

Tool: `accept\_quote`

Historia: HU-021 [MCP]

Descripción: Aceptar una cotización de receta y crear pedido vinculado para pago.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "recetaId": {
      "type": "string",
      "description": "ID de la receta cotizada a aceptar"
    }
  },
  "required": ["recetaId"]
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "pedido": {
      "type": "object",
      "properties": {
        "_id": { "type": "string" },
        "estado": { "type": "string", "enum": ["PENDIENTE"] },
        "total": { "type": "number" },
        "items": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "nombre": { "type": "string" },
              "precioSnapshot": { "type": "number" },
              "cantidad": { "type": "integer" }
            }
          }
        }
      },
      "metodoEntrega": { "type": "string" }
    },
    "checkoutUrl": {
      "type": "string",
      "format": "uri",
      "description": "URL de Stripe Checkout para pagar la receta"
    }
  }
}
```

```
}  
}
```

Endpoint: `POST /api/recetas/:id/aceptar` (nuevo endpoint MCP)

Auth: `User JWT` (dueño de la receta)

Rate Limit: 10 req/min por usuario

Idempotencia: Sí (aceptar receta ya aceptada retorna el pedido existente)

Errores:

Código	Significado
401	No autenticado
403	No es dueño de la receta
404	Receta no encontrada
409	Receta no está en estado COTIZADA
500	Error creando sesión Stripe

Tool: `admin\_list\_cakes`

Historia: HU-023 [MCP]

Descripción: Listar todos los pasteles del catálogo (admin) con búsqueda y paginación.

Input Schema:

```
json  
{  
  "type": "object",  
  "properties": {  
    "search": {  
      "type": "string",  
      "description": "Búsqueda por nombre o descripción"  
    },  
    "page": {  
      "type": "number",  
      "default": 1  
    },  
    "limit": {  
      "type": "number",  
      "default": 20,  
    },  
  },  
}
```



```

        "maximum": 100
    }
},
"additionalProperties": false
}

```

Output Schema:

```

json
{
  "type": "object",
  "properties": {
    "pasteles": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "_id": { "type": "string" },
          "nombre": { "type": "string" },
          "descripcion": { "type": "string" },
          "precio": { "type": "number" },
          "imagen": { "type": "string" },
          "categoria": { "type": "string" },
          "disponible": { "type": "boolean" },
          "createdAt": { "type": "string", "format": "date-time" }
        }
      }
    },
    "total": { "type": "number" },
    "page": { "type": "number" },
    "totalPages": { "type": "number" }
  }
}

```

Endpoint: `GET /api/admin/pasteles?search=chocolate&page=1`

Auth: `Admin JWT` (verifyAdmin + role check)

Rate Limit: 120 req/min por admin

Idempotencia: GET (seguro)

Errores:

Código	Significado
401	No autenticado
403	Intentando acceder a pedidos de otro usuario sin rol admin

Tool: `admin\_update\_order\_status`

Historia: HU-028 [MCP]

Descripción: Actualizar el estado de un pedido (solo transiciones válidas).

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "pedidoId": {
      "type": "string",
      "description": "ID del pedido a actualizar"
    },
    "nuevoEstado": {
      "type": "string",
      "enum": ["PENDIENTE", "PAGADO", "PREPARANDO", "LISTO", "EN_CAMINO", "ENTREGADO", "CANCELADO"],
      "description": "Nuevo estado del pedido"
    }
  },
  "required": ["pedidoId", "nuevoEstado"]
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "pedido": {
      "type": "object",
      "properties": {
        "_id": { "type": "string" },
        "estado": { "type": "string" },
        "updatedAt": { "type": "string", "format": "date-time" }
      }
    },
    "transicionesValidas": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Estados disponibles desde el estado actual"
    }
  }
}
```

Endpoint: `PUT /api/admin/pedidos/:id/status`

Auth: `Admin JWT`

Rate Limit: 120 req/min por admin

Idempotencia: No idempotente (cada cambio registra timestamp)

Transiciones válidas:

PENDIENTE → PAGADO, CANCELADO
PAGADO → PREPARANDO, CANCELADO
PREPARANDO → LISTO, CANCELADO
LISTO → EN_CAMINO, CANCELADO
EN_CAMINO → ENTREGADO
ENTREGADO → (terminal)
CANCELADO → (terminal)

Errores:

Código	Significado
400	Transición de estado no válida
401	No autenticado
403	No tiene rol admin
404	Pedido no encontrado

Tool: `admin\_quote\_recipe`

Historia: HU-030 [MCP]

Descripción: Cotizar una receta personalizada ingresando un precio y cambiando su estado.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "recetaId": {
      "type": "string",
      "description": "ID de la receta a cotizar"
    },
    "cotizacion": {
      "type": "number",
      "minimum": 1,
      "description": "Precio cotizado en MXN"
    }
  }
}
```

```

    },
    "required": ["recetaId", "cotizacion"]
  }
}

```

Output Schema:

```

json
{
  "type": "object",
  "properties": {
    "receta": {
      "type": "object",
      "properties": {
        "_id": { "type": "string" },
        "estado": { "type": "string", "enum": ["COTIZADA"] },
        "cotizacion": { "type": "number" },
        "updatedAt": { "type": "string", "format": "date-time" }
      }
    }
  }
}

```

Endpoint: `PUT /api/admin/recetas/:id/cotizar`

Auth: `Admin JWT`

Rate Limit: 60 req/min por admin

Idempotencia: No (cada cotización reemplaza la anterior)

Errores:

Código	Significado
400	Cotización inválida (≤ 0)
401	No autenticado
403	No tiene rol admin
404	Receta no encontrada
409	Receta no está en estado PENDIENTE o REVISANDO

Tool: `admin\_get\_dashboard\_stats`

Historia: HU-022 [MCP]

Descripción: Obtener estadísticas del dashboard administrativo en tiempo real.

Input Schema:

```
json
{
  "type": "object",
  "properties": {},
  "additionalProperties": false
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "pedidosHoy": { "type": "number" },
    "recetasPendientes": { "type": "number" },
    "productos": { "type": "number" },
    "ingresosMes": { "type": "number" },
    "recentPedidos": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "_id": { "type": "string" },
          "estado": { "type": "string" },
          "total": { "type": "number" },
          "createdAt": { "type": "string", "format": "date-time" }
        }
      }
    }
  }
}
```

Endpoint: `GET /api/admin/dashboard`

Auth: `Admin JWT`

Rate Limit: 30 req/min por admin (cálculos en tiempo real)

Idempotencia: GET (seguro, pero datos cambian en tiempo real)

Errores:

Código	Significado
401	No autenticado
403	No tiene rol admin

Tool: `handle\_stripe\_webhook`

Historia: HU-015 [MCP]

Descripción: Recibir y procesar eventos de Stripe (checkout.session.completed).

Input Schema (Stripe envía esto):

```
json
{
  "type": "object",
  "properties": {
    "id": { "type": "string" },
    "type": { "type": "string" },
    "data": {
      "type": "object",
      "properties": {
        "object": {
          "type": "object",
          "properties": {
            "id": { "type": "string" },
            "customer_details": {
              "type": "object",
              "properties": {
                "email": { "type": "string" },
                "name": { "type": "string" }
              }
            }
          }
        },
        "amount_total": { "type": "number" },
        "metadata": {
          "type": "object",
          "properties": {
            "pedidoId": { "type": "string" }
          }
        }
      }
    }
  }
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "received": { "type": "boolean", "enum": [true] }
  }
}
```

```
}
```

Endpoint: `POST /api/webhook/stripe`

Auth: `Webhook Secret (HMAC SHA256)` - No usa JWT. Valida firma con `stripe-signature` header.

Rate Limit: Exento (Stripe maneja su propio retry)

Idempotencia: Sí natural (Stripe envía `event.id` único; se ignora duplicado)

Proceso:

1. Validar firma HMAC con `STRIPE\_WEBHOOK\_SECRET`
2. Si evento es `checkout.session.completed`:
 - Extraer `session.id` (stripeSessionId)
 - Buscar pedido por `stripeSessionId`
 - Actualizar estado a `PAGADO`
 - Enviar email factura via Resend
3. Responder `{received: true}`

Errores:

Código	Significado
400	Firma HMAC inválida
400	Payload inválido

Tool: `admin\_search\_users`

Historia: HU-068 [MCP]

Descripción: Buscar y gestionar usuarios del sistema (admin).

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "search": {
      "type": "string",
      "description": "Búsqueda por email o nombre"
    },
    "role": {
      "type": "string",
      "enum": ["cliente", "admin", "superadmin"],
      "description": "Filtrar por rol"
    },
    "page": {
```

```

        "type": "number",
        "default": 1
    },
    "limit": {
        "type": "number",
        "default": 20,
        "maximum": 100
    }
},
"additionalProperties": false
}

```

Output Schema:

```

json
{
  "type": "object",
  "properties": {
    "usuarios": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "id": { "type": "string" },
          "email": { "type": "string" },
          "nombre": { "type": "string" },
          "role": { "type": "string" },
          "lastSignIn": { "type": "string", "format": "date-time" },
          "createdAt": { "type": "string", "format": "date-time" }
        }
      }
    },
    "total": { "type": "number" },
    "page": { "type": "number" },
    "totalPages": { "type": "number" }
  }
}

```

Endpoint: `GET /api/admin/usuarios?search=user@email.com&role=cliente`

Auth: `SuperAdmin JWT` (solo superadmin puede gestionar usuarios)

Rate Limit: 60 req/min por superadmin

Idempotencia: GET (seguro)

Errores:

Código	Significado
--------	-------------

401	No autenticado
-----	----------------

Tool: `export\_orders\_csv`

Historia: HU-033 [MCP]

Descripción: Exportar pedidos a CSV para contabilidad y reportes.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "fechaInicio": {
      "type": "string",
      "format": "date",
      "description": "Fecha inicio rango (YYYY-MM-DD)"
    },
    "fechaFin": {
      "type": "string",
      "format": "date",
      "description": "Fecha fin rango (YYYY-MM-DD)"
    },
    "estado": {
      "type": "string",
      "enum": ["PENDIENTE", "PAGADO", "PREPARANDO", "LISTO", "EN_CAMINO", "ENTREGADO", "CANCELADO"],
      "description": "Filtrar por estado"
    }
  },
  "additionalProperties": false
}
```

Output Schema:

```
json
{
  "type": "string",
  "format": "binary",
  "description": "Archivo CSV con columnas: ID, Fecha, Email, Items, Subtotal, IVA, Total, Estado, Método Entrega"
}
```

Endpoint: `GET /api/admin/pedidos/export?fechaInicio=2024-01-01&fechaFin=2024-01-31&format=csv`

Auth: `Admin JWT`

Rate Limit: 5 req/min por admin (operación pesada)

Idempotencia: GET (seguro)

Errores:

Código	Significado
401	No autenticado
403	No tiene rol admin
422	Rango de fechas inválido

Tool: `health\_check`

Historia: HU-036 [MCP]

Descripción: Verificar salud completa del sistema: backend, MongoDB, dependencias externas.

Input Schema:

```
json
{
  "type": "object",
  "properties": {
    "deep": {
      "type": "boolean",
      "default": false,
      "description": "Si true, verifica todas las dependencias (MongoDB, Stripe, Cloudinary, Clerk)"
    }
  },
  "additionalProperties": false
}
```

Output Schema:

```
json
{
  "type": "object",
  "properties": {
    "status": { "type": "string", "enum": ["ok", "degraded", "down"] },
    "timestamp": { "type": "string", "format": "date-time" },
    "uptime": { "type": "number", "description": "Segundos desde último restart" },
    "checks": {
      "type": "array",
      "items": {
        "type": "object",

```

```

    "properties": {
      "name": { "type": "string" },
      "status": { "type": "string", "enum": ["ok", "fail"] },
      "latencyMs": { "type": "number" },
      "error": { "type": "string" }
    }
  }
}
}
}

```

Endpoint: `GET /health?deep=true`

Auth: Público (sin auth)

Rate Limit: 30 req/min (proteger de scraping)

Idempotencia: GET (seguro)

Errores: Ninguno (siempre retorna 200 con status ok/degraded/down)

Tool: `admin\_export\_pasteles`

Historia: HU-026 [MCP] (extensión)

Descripción: Exportar catálogo completo de pasteles a CSV/JSON para sincronización con sistemas externos.

Input Schema:

```

json
{
  "type": "object",
  "properties": {
    "format": {
      "type": "string",
      "enum": ["csv", "json"],
      "default": "csv"
    }
  },
  "additionalProperties": false
}

```

Output Schema:

```

json
{
  "type": "string",
  "format": "binary",
  "description": "Archivo CSV o JSON con todos los pasteles"
}

```

Endpoint: `GET /api/admin/pasteles/export?format=csv`

Auth: `Admin JWT`

Rate Limit: 5 req/min por admin

Idempotencia: GET (seguro)

Errores:

Código	Significado
401	No autenticado
403	No tiene rol admin

Mapecto: Historias ↔ Tool MCP ↔ Auth ↔ Capability

Tabla de Mapecto MCP

#	Tool MCP	Historia	Auth Requerida	Capacidad del Agente
1	search_cakes	HU-001 [MCP]	API Key + User JWT	Buscar catálogo con filtros: categoría, precio min/max, disponibilidad, texto libre, paginación
2	get_cake_detail	HU-002 [MCP]	API Key + User JWT	Obtener detalle completo: descripción, imagen CDN, precio, stock, timestamps
3	add_to_cart	HU-004 [MCP]	User JWT	Mutar carrito: agregar pastel con cantidad, validar stock, retornar totales actualizados

4	<code>get_cart</code>	HU-008 [MCP]	User JWT	Leer carrito actual: items, subtotales, validación stock en tiempo real, items no disponibles
5	<code>create_order</code>	HU-014 [MCP]	User JWT	Crear pedido: validar items, snapshot precios, crear sesión Stripe, retornar checkoutUrl
6	<code>list_orders</code>	HU-016 [MCP]	User JWT	Listar pedidos: paginación, filtro estado, rango fechas, ordenación descendente
7	<code>get_order</code>	HU-017 [MCP]	User JWT	Detalle pedido: items con precio snapshot, tracking, factura email, timeline estados
8	<code>submit_recipe</code>	HU-019 [MCP]	User JWT	Enviar receta: upload archivo Cloudinary + crear receta con nota, personas, metadata
9	<code>list_my_recipes</code>	HU-020 [MCP]	User JWT	Listar mis recetas: filtro por estado, ver cotizaciones, historial comunicaciones
10	<code>accept_quote</code>	HU-021 [MCP]	User JWT	Aceptar cotización: crea pedido vinculado, retorna checkoutUrl para pago
11	<code>admin_list_cakes</code>	HU-023 [MCP]	Admin JWT	CRUD completo: listar, crear, actualizar, eliminar, toggle disponibilidad
12	<code>admin_update_order_statuses</code>	HU-028 [MCP]	Admin JWT	Cambiar estado: transiciones válidas, notas internas, notificaciones
13	<code>admin_quote_recipe</code>	HU-030 [MCP]	Admin JWT	Cotizar receta: set precio, cambiar a COTIZADA, notificar cliente
14	<code>admin_get_dashboard_stats</code>	HU-022 [MCP]	Admin JWT	KPIs tiempo real: ventas día/mes, pedidos por estado, recetas pendientes, conversión
15	<code>handle_stripe_webhook</code>	HU-015 [MCP]	Webhook Secret (HMAC)	Verificar firma, idempotencia, confirmar pago, emitir evento dominio

16	<code>admin_search_users</code>	HU-068 [MCP]	SuperAdmin JWT	Listar usuarios, filtrar por rol, paginación, asignar/revocar roles
17	<code>export_orders_csv</code>	HU-033 [MCP]	Admin JWT	Generar CSV contable: fecha, cliente, items, totales, impuestos, estado
18	<code>health_check</code>	HU-036 [MCP]	Público	Check dependencias: MongoDB, Stripe, Cloudinary, Clerk, latencia p95
19	<code>admin_export_pasteles</code>	HU-026 [MCP]	Admin JWT	Exportar catálogo completo a CSV/JSON para sincronización

Resumen por Nivel de Auth

Nivel Auth	Tools	Ejemplo
Público (sin auth)	1	<code>health_check</code>
User JWT	6	<code>add_to_cart</code> , <code>get_cart</code> , <code>list_orders</code> , <code>get_order</code> , <code>submit_recipe</code> , <code>list_my_recipes</code>
API Key + User JWT	4	<code>search_cakes</code> , <code>get_cake_detail</code> , <code>create_order</code> , <code>accept_quote</code>
Admin JWT	6	<code>admin_list_cakes</code> , <code>admin_update_order_status</code> , <code>admin_quote_recipe</code> , <code>admin_get_dashboard_stats</code> , <code>export_orders_csv</code> , <code>admin_export_pasteles</code>
SuperAdmin JWT	1	<code>admin_search_users</code>
Webhook Secret (HMAC)	1	<code>handle_stripe_webhook</code>
Total	19	

Ejemplo de Uso en Claude Code

```
bash
# Buscar pasteles de chocolate
claude mcp call search_cakes --categoria chocolate --precioMin 400

# Ver detalle de pastel
claude mcp call get_cake_detail --pastelId "6651a2b3c4d5e6f7a8b9c0d1"

# Agregar al carrito
claude mcp call add_to_cart --pastelId "6651a2b3c4d5e6f7a8b9c0d1" --cantidad 2

# Ver carrito
claude mcp call get_cart

# Crear pedido
claude mcp call create_order \
  --email "cliente@email.com" \
  --items '[{"pastelId":"6651a2b3c4d5e6f7a8b9c0d1","cantidad":2}]' \
  --metodoEntrega "DOMICILIO" \
  --direccionEnvio "Av. Principal 123, CDMX" \
  --telefono "5512345678"

# Listar mis pedidos
claude mcp call list_orders --estado PAGADO

# Enviar receta personalizada
claude mcp call submit_recipe \
  --nota "Pastel de 3 pisos con fresas y crema, decoración floral" \
  --personas 30 \
  --archivoUrl "https://res.cloudinary.com/xxx/receta.pdf"

# Admin: ver dashboard
claude mcp call admin_get_dashboard_stats

# Admin: cotizar receta
claude mcp call admin_quote_recipe --recetaId "6651a2b3c4d5e6f7a8b9c0d2"
--cotizacion 1500

# Admin: cambiar estado pedido
claude mcp call admin_update_order_status --pedidoId "6651a2b3c4d5e6f7a8b9c0d3"
--nuevoEstado PREPARANDO

# Admin: exportar pedidos
claude mcp call export_orders_csv --fechaInicio "2024-01-01" --fechaFin
"2024-01-31"

# Health check
claude mcp call health_check --deep true
```

Ejemplo de Uso en Codex

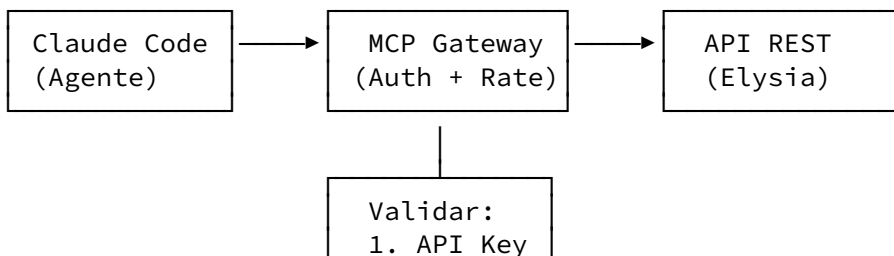
```
python
# pip install openai (o SDK MCP equivalente)

import openai

# Configurar cliente MCP
client = openai.OpenAI(
    base_url="https://api.dolceatelier.com/mcp/v1",
    api_key="mcp_sk_live_XXXXX"
)

# Buscar pasteles
result = client.chat.completions.create(
    model="gpt-4",
    tools=[{
        "type": "function",
        "function": {
            "name": "search_cakes",
            "description": "Buscar pasteles del catálogo",
            "parameters": {
                "type": "object",
                "properties": {
                    "categoria": {"type": "string", "enum": ["chocolate",
"vainilla", "frutas", "especial"]},
                    "precioMin": {"type": "number"},
                    "precioMax": {"type": "number"}
                }
            }
        }
    }],
    messages=[{"role": "user", "content": "Muéstrame los pasteles de chocolate
entre $400 y $600"}]
)
```

Flujo de Autenticación MCP



- | |
|--|
| <ol style="list-style-type: none">2. JWT3. Rate4. RBAC |
|--|

Headers requeridos:

X-API-Key: mcp_sk_live_XXXXX # Identifica plan y cuota
Authorization: Bearer eyJhbG... # JWT de Clerk (usuario final)
Content-Type: application/json

Respuesta de error auth:

```
json
{
  "error": {
    "code": "UNAUTHORIZED",
    "message": "API Key inválida o inactiva",
    "requestId": "req-abc123"
  }
}
```

Resumen Ejecutivo - Historias de Usuario Dolce Atelier

Contexto

Dolce Atelier es una plataforma B2C de pasteles artesanales que permite:

- Comprar pasteles del catálogo
- Solicitar pasteles personalizados mediante recetas
- Elegir entre entrega a domicilio o retiro en tienda

Stack: Next.js 14 (Frontend) + Elysia/Bun (Backend) + MongoDB + Clerk (Auth) + Stripe (Pagos) + Cloudinary (Imágenes) + Resend (Email)

Objetivo del documento: Mapear todas las funcionalidades del MVP actual y las necesarias para alcanzar estado **production-ready**, con énfasis en funcionalidades que se expondrán vía **MCP (Model Context Protocol)** para agentes IA.

Estado Actual del MVP

✅ Implementado (40 historias)

Área	Historias	Estado
Catálogo Público	HU-001, HU-002, HU-003	✅ Completo (SSR, filtros, paginación)
Carrito de Compras	HU-004, HU-005, HU-006, HU-007, HU-008	✅ Completo (Zustand, localStorage, merge)
Checkout + Pagos	HU-009 a HU-015	✅ Completo (Stripe Checkout + Webhook)
Pedidos Cliente	HU-016, HU-017	✅ Completo (historial, detalle, filtros)
Recetas	HU-018 a HU-021	⚠ Backend listo, Frontend placeholder
Admin Dashboard	HU-022	✅ Completo (KPIs en tiempo real)
Admin CRUD	HU-023 a HU-030	✅ Completo (pasteles, pedidos, recetas)
Auth + Seguridad	HU-031 a HU-034	✅ Completo (Clerk, roles, middleware)
Infraestructura	HU-035 a HU-040	✅ Completo (seed, health, dev)

❌ No implementado (excluido del MVP)

Área	Detalle
Frontend Recetas	Solo placeholder "Próximamente"
Tests	Ninguno (unit, integration, E2E)
CI/CD	No configurado
Observabilidad	Solo console.log

Rate Limiting	No implementado
Accesibilidad	No auditado
i18n	Solo español hardcodeado

Production-Ready: Lo que Falta (40 historias nuevas)

Categoría	Historias	Prioridad
Seguridad/Authz	HU-041 a HU-045	 Crítica
Pagos/Webhooks	HU-046 a HU-049	 Crítica
Observabilidad	HU-050 a HU-054	 Alta
Manejo Errores	HU-055 a HU-057	 Alta
Testing	HU-058 a HU-062	 Alta
CI/CD	HU-063 a HU-066	 Alta
RBAC/Permisos	HU-067 a HU-070	 Media
Rate Limiting	HU-071 a HU-072	 Media
Backups/DR	HU-073 a HU-075	 Media
Accesibilidad	HU-076 a HU-078	 Normal
i18n	HU-079 a HU-080	 Normal

MCP: Herramientas para Agentes IA (19 tools)

Concepto

El **Model Context Protocol (MCP)** expone la API REST de Dolce Atelier como herramientas tipadas que agentes IA (Claude Code, Codex, Cursor) pueden consumir programáticamente bajo autenticación `API Key +`

User JWT`.

Herramientas Definidas

Tool	Historia	Auth	Uso por Agente
<code>search_cakes</code>	HU-001	API Key + JWT	Buscar pasteles por categoría, precio, texto
<code>get_cake_detail</code>	HU-002	API Key + JWT	Ver detalle completo de un pastel
<code>add_to_cart</code>	HU-004	User JWT	Agregar pastel al carrito
<code>get_cart</code>	HU-008	User JWT	Ver carrito con totales
<code>create_order</code>	HU-014	User JWT	Crear pedido y obtener URL de pago
<code>list_orders</code>	HU-016	User JWT	Listar pedidos con filtros
<code>get_order</code>	HU-017	User JWT	Ver detalle de pedido
<code>submit_recipe</code>	HU-019	User JWT	Enviar receta personalizada
<code>list_my_recipes</code>	HU-020	User JWT	Ver mis recetas
<code>accept_quote</code>	HU-021	User JWT	Aceptar cotización y pagar
<code>admin_list_cakes</code>	HU-023	Admin JWT	CRUD completo de pasteles
<code>admin_update_order_statuses</code>	HU-028	Admin JWT	Cambiar estado de pedido
<code>admin_quote_recipe</code>	HU-030	Admin JWT	Cotizar receta
<code>admin_get_dashboard_stats</code>	HU-022	Admin JWT	KPIs del negocio
<code>handle_stripe_webhook</code>	HU-015	HMAC Secret	Procesar webhook de Stripe
<code>admin_search_users</code>	HU-068	SuperAdmin JWT	Gestionar usuarios
<code>export_orders_csv</code>	HU-033	Admin JWT	Exportar pedidos a CSV
<code>health_check</code>	HU-036	Público	Verificar salud del sistema
<code>admin_export_pasteles</code>	HU-026	Admin JWT	Exportar catálogo

Autenticación MCP

X-API-Key: mcp_sk_live_xxxxx # Plan (free/pro/enterprise)
Authorization: Bearer eyJhbG... # JWT de Clerk (usuario)

Ejemplo en Claude Code

```
bash
# Buscar pasteles de chocolate
claude mcp call search_cakes --categoria chocolate

# Crear pedido
claude mcp call create_order \
  --email "cliente@email.com" \
  --items '[{"pastelId":"abc123","cantidad":2}]' \
  --metodoEntrega "DOMICILIO"
```

Criterios de Validación

Criterio	Estado
Numeración global continua HU-001 a HU-080	
≥ 30 historias nuevas production-ready	 40
≥ 15 historias [MCP] del total	 36
Formato "Como/Quiero/Para"	
2-4 criterios de aceptación por historia	
Tools MCP con JSON Schema	
Auth documentada por tool	
Español técnico	

Próximos Pasos

1. **Revisión grupal** - Discutir cuáles historias se implementarán primero
2. **Priorización** - Criterios de MoSCoW (Must/Should/Could/Won't)
3. **Estimación** - Story points por historia
4. **Sprint Planning** - Definir sprints para implementar historias seleccionadas
5. **Implementación MCP** - Construir gateway MCP con las 19 tools definidas